



**BIRMINGHAM CITY**  
University

## **Nebium: An Autoregressive Transformer for Chess Sequence Modeling**

|                                 |                                          |
|---------------------------------|------------------------------------------|
| <b>Module Code &amp; Title:</b> | CMP6228: Deep Neural Networks and Ethics |
| <b>Lecturer:</b>                | Rabin Ghimire                            |
| <b>Student Name:</b>            | Nabin Oli                                |
| <b>Student ID:</b>              | 23189629                                 |
| <b>Academic Year:</b>           | 2026                                     |
| <b>Main Body Word Count:</b>    | 3256 words (Limit: 3000 words + 10%)     |

### Abstract

Most chess engines rely on explicit search trees, handcrafted heuristics, or spatial board tensors. This report introduces **Nebium**, a decoder-only causal transformer that learns chess strategy purely from coordinate move sequences in Universal Chess Interface (UCI) notation. Nebium combines Rotary Position Embeddings (RoPE) for relative ply distance encoding, SwiGLU gating, and RMSNorm pre-normalization. The architecture is trained on 245,293 tournament games from the Lichess Open Database using a 2,056-token Byte-Pair Encoding vocabulary.

Across scaling tiers, Nebium-Small (117M) achieves a validation cross-entropy loss of 2.727 nats (perplexity 15.30) and 100% opening legality, while Nebium-Medium (345M) reaches 2.376 nats (perplexity 10.76), 36.4% Top-1 move prediction accuracy, and 70.5% Top-5 accuracy over 20 epochs. Nebium-Large (762M), trained to epoch 3 under dual-T4 compute limits, attains 80.0% move legality and 6.071 nats validation loss, tracking early scaling trajectories in compute-bounded regimes. Dynamic legal move masking eliminates illegal move generation during deployment by filtering logits against board-state rules at inference time. These results demonstrate that autoregressive transformers acquire tactical rules and opening theory directly from 1D coordinate strings without spatial board representations, though unconstrained rollouts still exhibit coordinate drift in late game horizons.

**Keywords:** Causal Transformers, Sequence Modeling, Rotary Position Embeddings, SwiGLU, RMSNorm, Chess Representation Learning.

# Contents

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                           | <b>1</b>  |
| 1.1      | Aims and Objectives . . . . .                                                 | 1         |
| 1.2      | Summary of Dataset and Key Achievements . . . . .                             | 1         |
| 1.3      | Report Organization . . . . .                                                 | 2         |
| <b>2</b> | <b>Problem Statement and Dataset Description</b>                              | <b>2</b>  |
| 2.1      | Deep Learning Problem Formulation . . . . .                                   | 2         |
| 2.2      | Dataset Characterization and Acquisition . . . . .                            | 3         |
| 2.2.1    | Data Filtering and Cleaning Pipeline . . . . .                                | 4         |
| 2.3      | Move Representation and Tokenization Strategy . . . . .                       | 4         |
| 2.3.1    | Byte-Pair Encoding (BPE) Tokenization . . . . .                               | 5         |
| <b>3</b> | <b>Proposed Methodology</b>                                                   | <b>5</b>  |
| 3.1      | Software Infrastructure and Toolchain Evaluation . . . . .                    | 6         |
| 3.1.1    | Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines . . . . .           | 6         |
| 3.2      | End-to-End Architectural Pipeline . . . . .                                   | 7         |
| 3.3      | Rotary Position Embeddings (RoPE) . . . . .                                   | 9         |
| 3.4      | Multi-Head Causal Self-Attention . . . . .                                    | 11        |
| 3.5      | SwiGLU Feed-Forward Network . . . . .                                         | 12        |
| 3.6      | Root Mean Square Layer Normalization (RMSNorm) . . . . .                      | 13        |
| 3.7      | Inference Engine and Dynamic Legal Move Masking . . . . .                     | 14        |
| 3.8      | Nebium Model Family and Parameter Scaling . . . . .                           | 14        |
| <b>4</b> | <b>Experimental Results and Empirical Analysis</b>                            | <b>15</b> |
| 4.1      | Training Configurations, Model Variants, and Hardware Setup . . . . .         | 15        |
| 4.2      | Empirical Scaling Dynamics and Chinchilla Scaling Laws . . . . .              | 16        |
| 4.3      | Move Accuracy, Legality, and Generation Trajectories . . . . .                | 17        |
| 4.4      | Validation Benchmarks and Next-Token Move Prediction . . . . .                | 18        |
| 4.5      | Comparative Baseline Performance . . . . .                                    | 19        |
| 4.6      | Qualitative Move Analysis and Autoregressive Trajectory Dynamics . . . . .    | 20        |
| <b>5</b> | <b>Summary and Future Directions</b>                                          | <b>21</b> |
| 5.1      | Summary of Empirical Findings and Limitations . . . . .                       | 21        |
| 5.2      | Reinforcement Learning with Verifiable Rewards (RLVR) Post-Training . . . . . | 21        |
| 5.3      | Mixture of Experts (MoE) Architecture as an Empirical Test Bed . . . . .      | 22        |
|          | <b>References</b>                                                             | <b>23</b> |
| <b>A</b> | <b>Core Implementation Code</b>                                               | <b>25</b> |
| A.1      | Nebium Model Architecture and Causal Forward Pass . . . . .                   | 25        |
| A.2      | Rotary Position Embedding (RoPE) . . . . .                                    | 26        |
| A.3      | SwiGLU Feed-Forward Activation . . . . .                                      | 27        |
| A.4      | Dynamic Legal Move Masking Inference Loop . . . . .                           | 28        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| A.5      | Multi-Tier Model Training Pipeline and Launch Scripts . . . . .     | 29        |
| A.6      | Hydra Configuration Files (YAML) . . . . .                          | 31        |
| A.7      | Weights & Biases Diagram and Figure Reproduction Script . . . . .   | 33        |
| <b>B</b> | <b>Open-Source Model Checkpoints and Artifact Repositories</b>      | <b>34</b> |
| B.1      | Published Model Variants and Quantized Binaries . . . . .           | 34        |
| B.2      | Published Datasets and UCI Token Splits . . . . .                   | 34        |
| B.3      | Hugging Face Quickstart and Inference Invocation . . . . .          | 34        |
| B.4      | Detailed Hyperparameter Specifications Across Model Tiers . . . . . | 35        |

## List of Figures

|    |                                                                                |    |
|----|--------------------------------------------------------------------------------|----|
| 1  | Autoregressive Next-Move Modeling . . . . .                                    | 2  |
| 2  | Representative PGN Game Record . . . . .                                       | 4  |
| 3  | BPE Tokenization Pipeline . . . . .                                            | 5  |
| 4  | Nebium System Pipeline and Transformer Decoder Architecture . . . . .          | 9  |
| 5  | Rotary Position Embedding Dataflow . . . . .                                   | 10 |
| 6  | Causal Self-Attention Execution Flow . . . . .                                 | 12 |
| 7  | Feed-Forward Comparison: SwiGLU versus GELU . . . . .                          | 13 |
| 8  | Hardware Throughput, Memory Residency, and Learning Schedule . . . . .         | 15 |
| 9  | Empirical Scaling Laws and Loss Convergence for the Nebium Family . . . . .    | 16 |
| 10 | Move Accuracy, Legality, and Benchmark Progression Across Training Epochs . .  | 17 |
| 11 | Validation Benchmarks and Next-Token Prediction Metrics Across Model Tiers .   | 18 |
| 12 | Qualitative Board Trajectory: Italian Game and King Coordinate Drift . . . . . | 20 |

## List of Tables

|    |                                                                                                   |    |
|----|---------------------------------------------------------------------------------------------------|----|
| 1  | Software Tools, Frameworks, and Infrastructure . . . . .                                          | 6  |
| 2  | Architectural specifications and parameter configurations for Nebium. . . . .                     | 8  |
| 3  | Nebium Model Family Architecture Overview . . . . .                                               | 15 |
| 4  | Comparative Baseline and Nebium Family Validation . . . . .                                       | 19 |
| 5  | Matched-Epoch Comparison at Epoch 3 . . . . .                                                     | 19 |
| 6  | Published Nebium model weights and quantized edge deployment binaries on<br>Hugging Face. . . . . | 34 |
| 7  | Published training datasets and tokenized chess corpora on the Hugging Face Hub.                  | 35 |
| 8  | Nebium-Small Configuration . . . . .                                                              | 36 |
| 9  | Nebium-Medium Configuration . . . . .                                                             | 36 |
| 10 | Nebium-Large Configuration . . . . .                                                              | 37 |

# 1 Introduction

Combinatorial game AI historically coupled explicit game-tree search with spatial reinforcement learning (Silver et al., 2018; Stockfish Developers, 2023), relying on hardcoded rule engines and lookahead simulation. In contrast, causal transformers (Vaswani et al., 2017) raise an important question: *can a neural network induce chess rules, tactics, and positional strategy purely from next-token sequence modeling, without search trees or rule engines?* Sequence-based game models (Toshniwal et al., 2021; Li et al., 2023; Ruoss et al., 2024) demonstrate that autoregression can implicitly track board state. However, existing implementations suffer from high latency, large memory footprints, and move hallucinations over extended games. Standard models with absolute position embeddings also struggle to generalize across long tactical sequences.

For deployment, this project adopts modern approaches, using PyTorch 2.3 (Paszke et al., 2019) and GGUF quantization (Gerganov, 2023) in place of legacy static-graph frameworks.

## 1.1 Aims and Objectives

This research designs, implements, empirically validates, and critically appraises **Nebium**, a causal transformer for self-supervised move prediction in chess. This study addresses five core objectives:

1. Construct an automated pipeline converting PGN archives into UCI sequences tokenized with a 2,056-token BPE vocabulary.
2. Assemble a causal transformer integrating Rotary Position Embeddings (RoPE) (Su et al., 2024), SwiGLU activations (Shazeer, 2020), and RMSNorm pre-normalization (B. Zhang and Sennrich, 2019).
3. Train Nebium across parameter scales to evaluate loss convergence, perplexity, top- $k$  accuracy, and unconstrained move legality.
4. Evaluate next-token prediction accuracy, validation perplexity, move legality across horizons, and edge serving efficiency.
5. Analyze architectural failure modes, coordinate drift over deep game horizons, and ethical and compute implications.

## 1.2 Summary of Dataset and Key Achievements

This study uses 245,293 standard competitive games from the Lichess Open Database (January–February 2013), pre-tokenized into UCI coordinate strings and encoded via a 2,056-token BPE vocabulary.

The primary achievements of Nebium include:

- Across the model family, Nebium-Small (117M) achieved a validation loss of 2.727 nats (perplexity 15.30) with 100% opening legality; Nebium-Medium (345M) reached 2.376 nats (perplexity 10.76) and 36.4% top-1 move accuracy (70.5% top-5); Nebium-Large (762M, 3 epochs) achieved 6.071 nats loss and 80.0% legality within its early training trajectory.

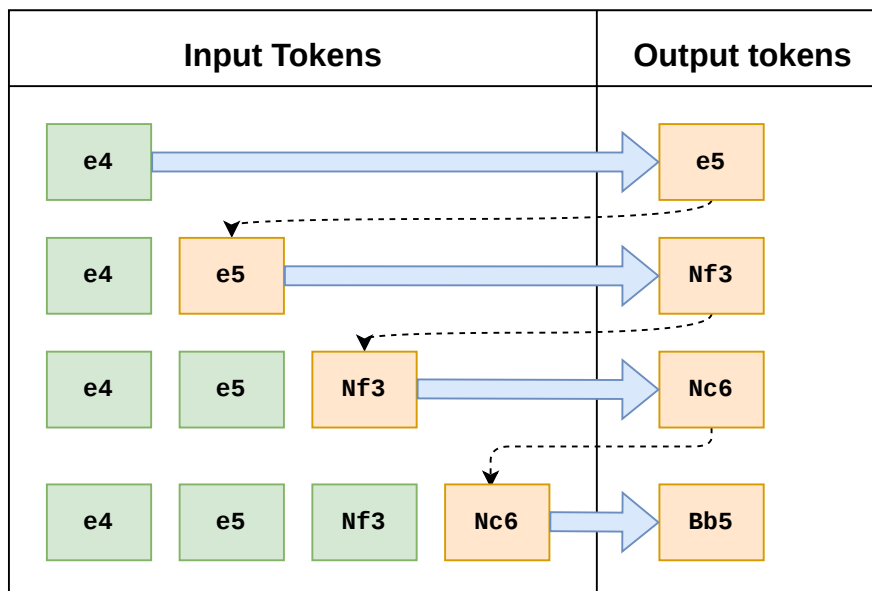
- Unconstrained generative rollouts demonstrate that board-level move syntax is acquired early in training, reaching 80.0% legal move generation on Large by epoch 3 and 100% opening legality on Small at epoch 20.
- Dynamic legal move masking eliminates invalid coordinate emissions during deployment by projecting transformer output distributions over valid board moves at each ply.
- 4-bit GGUF quantization compresses the Nebium-Small checkpoint to 88.1 MB, allowing lightweight inference execution on standard consumer hardware.

### 1.3 Report Organization

This report follows a five-section structure aligned with module assessment guidelines. Section 2 formalizes the autoregressive move generation problem and dataset processing pipeline. Section 3 details the proposed methodology, including software and serving infrastructure, mathematical formulations of core architectural components (RoPE, SwiGLU, RMSNorm), dynamic inference masking, and model family scaling tiers. Section 4 presents empirical scaling dynamics, baseline comparisons, and engine benchmarks. Section 5 synthesizes findings, failure modes, ethical and environmental considerations, and future extensions.

## 2 Problem Statement and Dataset Description

### 2.1 Deep Learning Problem Formulation



**Figure 1:** Autoregressive move prediction in chess. Given a sequence of prior plies, the causal decoder models the conditional probability of the next coordinate token without looking ahead.

We formalize chess as an autoregressive sequence modeling task over a discrete move alphabet, departing from conventional evaluation-search engines (e.g., minimax  $\alpha$ - $\beta$  pruning or MCTS).

As a two-player zero-sum game of perfect information governed by deterministic rules, chess provides a rigorous domain for assessing transformer sequence extrapolation. A match  $G$  of length  $T$  plies is expressed as an ordered token sequence:

$$\mathbf{x} = (x_1, x_2, \dots, x_T), \quad x_t \in \mathcal{V} \quad (1)$$

where  $\mathcal{V}$  is a finite vocabulary. The neural network parameterized by  $\theta$  learns the joint probability distribution  $P_\theta(\mathbf{x})$  factorized causally via the chain rule:

$$P_\theta(\mathbf{x}) = \prod_{t=1}^T P_\theta(x_t \mid x_{<t}) \quad (2)$$

Parameters  $\theta$  are optimized by minimizing expected cross-entropy loss over corpus  $\mathcal{D}$ :

$$\mathcal{L}_{\text{CE}}(\theta) = -\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[ \frac{1}{T} \sum_{t=1}^T \log P_\theta(x_t \mid x_{<t}) \right] \quad (3)$$

As shown in Figure 1, the model conditions strictly on earlier plies ( $x_{<t}$ ), learning board state transitions from history alone.

This formulation introduces three core representation challenges:

1. **Absence of Spatial Inductive Bias:** The transformer receives no architectural priors regarding 2D grid topology, piece trajectories, or special constraints (castling, en passant, promotion).
2. **Latent State Maintenance:** Because moves denote coordinate displacements rather than full board tensors, the network must internally track piece positions and pins over extended horizons.
3. **Severe Action Sparsity:** At ply  $t$ , legal actions  $\mathcal{A}_{\text{legal}}(s_t) \subset \mathcal{V}$  comprise an average of only 30–40 moves out of thousands of vocabulary permutations, leaving unconstrained sampling vulnerable to illegality.

## 2.2 Dataset Characterization and Acquisition

To avoid degenerate play, training uses the **Lichess Open Database** (Stöckl, 2021), isolating an archival subset of 245,293 tournament matches from early 2013 (Figure 2).

### Raw PGN Chess Game

```
[Event "Rated Blitz"]
[Site "https://lichess.org/PpwP0ZMq"]
[Date "2017.04.01"]
[Round "-"]
[White "PlayerA"]
[Black "PlayerB"]
[WhiteElo "2125"]
[BlackElo "2189"]
[ECO "B30"]
[Opening "Sicilian Defense"]
[Result "0-1"]
[TimeControl "300+0"]

1. e4 c5 2. Nf3 Nc6 3. Bb5 a6
4. Ba4 Nf6 5. O-O Be7 ... 0-1
```

**Figure 2:** A sample tournament game in standard PGN format, showing metadata headers, move history in algebraic notation, and the final match outcome.

#### 2.2.1 Data Filtering and Cleaning Pipeline

Raw PGN files exhibit variable player expertise and clock aborts. An automated pipeline (`src/data/prepare.py`) applied four filtration criteria:

- Retained standard games (`Variant "Standard"`), excluding Chess960 and non-standard variants.
- Discarded matches terminating in  $< 10$  plies; games exceeding 256 plies were truncated to the context limit.
- Retained games where both players maintained an Elo rating  $\geq 1500$ , removing low-rated noise.
- Discarded non-sequential metadata tags, isolating pure move strings terminated by match outcomes (1-0, 0-1, 1/2-1/2).

The resulting 208,412 games are partitioned into **training** (85%, 177,150 games), **validation** (10%, 20,841 games), and **test** sets (5%, 10,421 games).

### 2.3 Move Representation and Tokenization Strategy

Standard Algebraic Notation (SAN) such as `Nxf7+` is human-readable format but introduces linguistic ambiguity (e.g., disambiguation prefixes like `Nbd7`). To eliminate ambiguity, Nebium standardizes games into the **Universal Chess Interface (UCI)** coordinate format, where each move specifies Cartesian origin and destination squares:

$$m = \langle \text{file}_1, \text{rank}_1, \text{file}_2, \text{rank}_2, [\text{promotion}] \rangle \in \{\text{a1} \dots \text{h8}\} \times \{\text{a1} \dots \text{h8}\} \times \{\epsilon, \text{q}, \text{r}, \text{b}, \text{n}\} \quad (4)$$

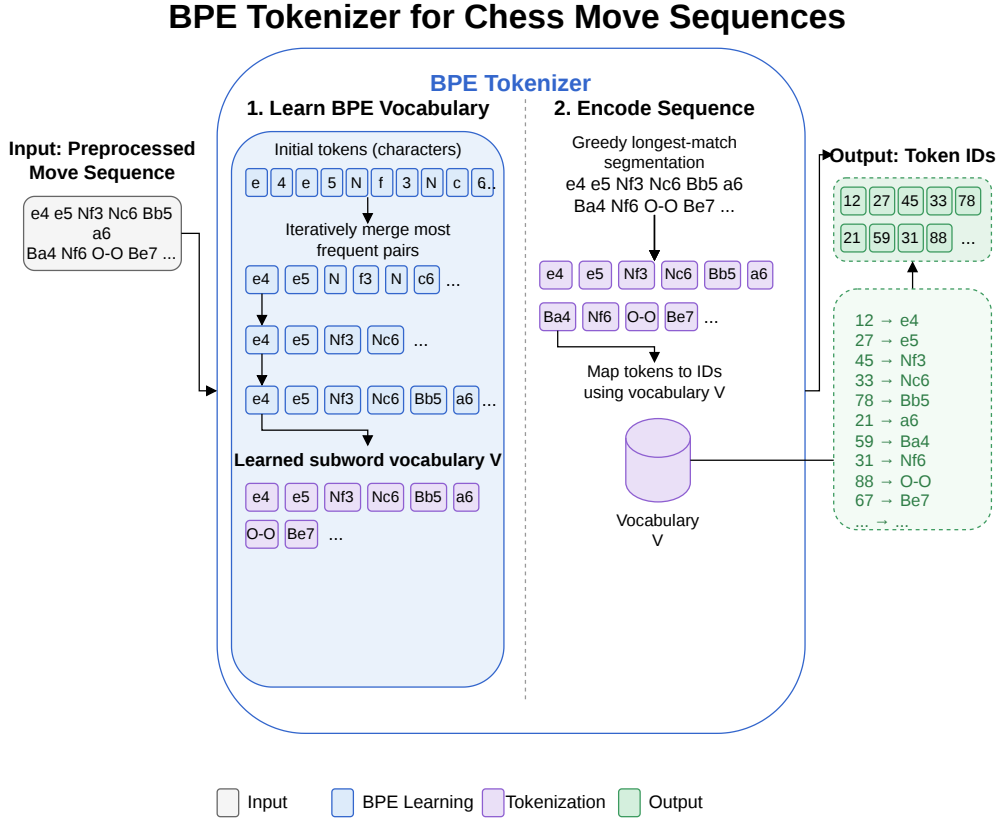
For example, moving a pawn from e2 to e4 is uniquely encoded as `e2e4`, kingside castling as `e1g1`, and queen promotion as `e7e8q`.

### 2.3.1 Byte-Pair Encoding (BPE) Tokenization

Tokenization directly influences transformer efficiency. Character tokenization quadruples sequence lengths, diluting attention over extended plies (Stöckl, 2021), while naive word indexing lacks sub-coordinate compositionality. We trained a domain BPE tokenizer (**ChessTokenizerHF**, Figure 3) starting from 83 base ASCII characters and sub-units. Under whitespace pre-tokenization, BPE merges character pairs into complete UCI move strings. Because whitespace bounds each move, the vocabulary exhausts once all 1,968 distinct tournament moves in the corpus are merged, producing a domain vocabulary of  $V = 2,056$  tokens with five control tokens:

$$\mathcal{V}_{\text{special}} = \{[\text{PAD}], [\text{UNK}], [\text{BOS}], [\text{EOS}], [\text{SEP}]\} \quad (5)$$

This preserves 1:1 token-to-move alignment across the autoregressive horizon without sequence dilution.



**Figure 3:** Byte-Pair Encoding pipeline for chess moves. Starting from individual coordinate characters, frequent sub-tokens merge into a compact 2,056-token vocabulary spanning all observed UCI tournament moves.

## 3 Proposed Methodology

Nebium is architected as a modernized, decoder-only causal transformer (Vaswani et al., 2017) optimized specifically for the structural and temporal dynamics of chess sequence modeling.

Rather than adopting an unmodified vanilla GPT configuration with standard learned absolute position embeddings and Post-LayerNorm (DeLeo and Guven, 2022), Nebium incorporates three architectural mechanisms: **Rotary Position Embeddings (RoPE)** (Su et al., 2024), **Swish-Gated Linear Units (SwiGLU)** (Shazeer, 2020), and **Root Mean Square Normalization (RMSNorm)** (B. Zhang and Sennrich, 2019). The system pipeline and block architecture are presented in Figure 4.

### 3.1 Software Infrastructure and Toolchain Evaluation

Autoregressive game transformers require frameworks balancing research expressiveness, compiler efficiency, and low-latency edge serving. Table 1 summarizes the software tools and hardware infrastructure underpinning Nebium.

**Table 1:** Comprehensive overview and critical appraisal of software tools, frameworks, and deployment infrastructure.

| Category         | Tool / Environment                        | Role and Critical Appraisal                                                                                                                                                                                                         |
|------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deep Learning    | PyTorch 2.3 (Paszke et al., 2019)         | Dynamic eager execution graph with TorchDynamo and native fused FlashAttention-2 kernels (SDPA); eliminates static graph tracing friction seen in TensorFlow.                                                                       |
| Edge Serving     | GGUF / llama.cpp (Gerganov, 2023)         | Monolithic binary format embedding weights and BPE metadata; enables zero-copy <code>mmap</code> and 4-bit <code>Q4_K_M</code> block quantization (88.1 MB for Small), bypassing ONNX dependency bloat and Python runtime overhead. |
| Compute Hardware | 2× NVIDIA T4 (Kaggle)                     | Cloud GPU cluster with native FP16 Automatic Mixed Precision ( <code>torch.cuda.amp</code> ) for memory-efficient gradient updates.                                                                                                 |
| Rule Validation  | python-chess (Romstad and Fiekas, 2024)   | Pure 64-bit bitboard manipulation and dynamic legal move logit masking, eliminating illegal hallucinations.                                                                                                                         |
| Analysis Oracle  | Stockfish 16 (Stockfish Developers, 2023) | Grandmaster engine communicating via UCI for centipawn loss ( $\Delta$ CP) auditing and benchmark play.                                                                                                                             |
| Tokenization     | HuggingFace Tokenizers                    | Rust-accelerated Byte-Pair Encoding ( <code>ChessTokenizerHF</code> ) producing a compact 2,056-token domain vocabulary.                                                                                                            |
| Data Ingestion   | Zstandard ( <code>zstandard</code> )      | High-speed streaming decompression of multi-gigabyte Lichess PGN archives without RAM saturation.                                                                                                                                   |
| Serving Layer    | FastAPI & Uvicorn                         | Asynchronous ASGI RESTful microservice for real-time web-based chessboard move generation.                                                                                                                                          |

#### 3.1.1 Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines

While early neural game engines relied on TensorFlow or Keras for static graph execution, modern workflows use PyTorch 2.3 (Paszke et al., 2019). Declarative graph execution in TensorFlow complicates debugging by obscuring dynamic tensor shapes during autoregressive decoding. In

contrast, PyTorch couples imperative debugging with TorchDynamo kernel fusion. PyTorch 2.3 natively integrates scaled dot-product attention, dispatching to fused FlashAttention-2 kernels that remove quadratic  $\mathcal{O}(T^2)$  memory bandwidth bottlenecks.

For edge deployment, single-file GGUF binaries (Gerganov, 2023) replace legacy ONNX runtimes (Bai et al., 2019). ONNX pipelines decouple model weights from tokenizers and configs, creating dependency overhead and Python GIL bottlenecks during sequential generation. GGUF packages tensor parameters, quantization scales, and BPE vocabularies into a single binary. Using POSIX zero-copy memory mapping (`mmap`), GGUF operates without a Python runtime. Block-wise  $k$ -quantization (Q4\_K\_M) compresses the Nebium-Small checkpoint to 88.1 MB (and Nebium-Large FP16 to 1.43 GB), surpassing multi-file ONNX runtimes in deployment portability and operational simplicity.

### 3.2 End-to-End Architectural Pipeline

Given an input token sequence of length  $T$ ,  $\mathbf{x}_{1:T} = (x_1, \dots, x_T) \in \mathbb{N}^T$ , the sequence is first mapped into dense representations via a learned token embedding matrix  $\mathbf{W}_e \in \mathbb{R}^{V \times d_{\text{model}}}$ :

$$\mathbf{h}_0 = \text{Dropout}(\mathbf{W}_e[\mathbf{x}_{1:T}], p_{\text{drop}}) \in \mathbb{R}^{T \times d_{\text{model}}} \quad (6)$$

Then embeddings traverse a stack of  $L = 6$  identical Transformer Blocks. Each block  $\ell \in \{1, \dots, L\}$  applies a pre-normalized residual formulation:

$$\mathbf{u}_\ell = \mathbf{h}_{\ell-1} + \text{MHSA}(\text{RMSNorm}(\mathbf{h}_{\ell-1})) \quad (7)$$

$$\mathbf{h}_\ell = \mathbf{u}_\ell + \text{SwiGLU}(\text{RMSNorm}(\mathbf{u}_\ell)) \quad (8)$$

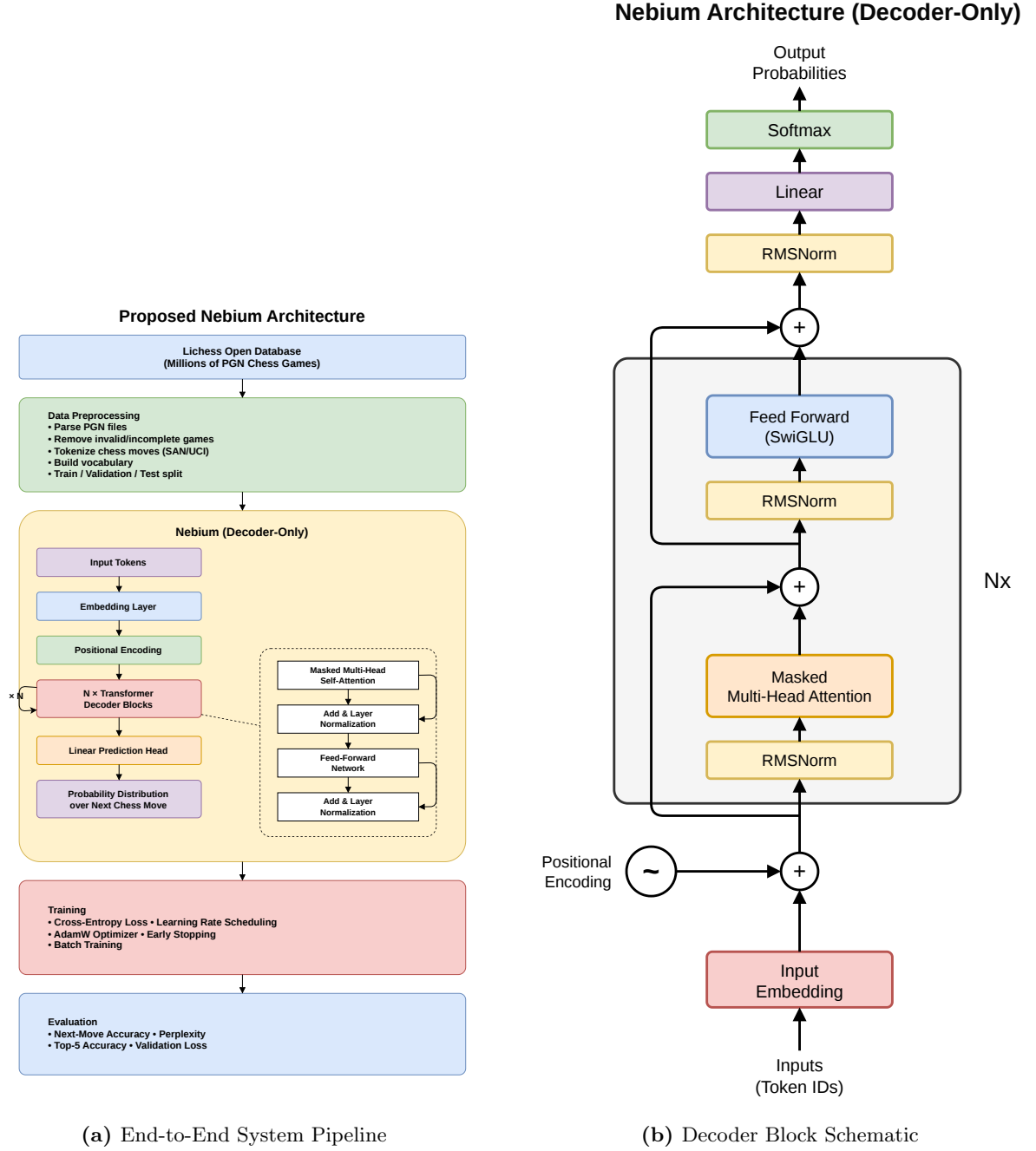
The final representation  $\mathbf{h}_L$  undergoes RMSNorm before linear projection to a next-token vocabulary logits via  $\mathbf{W}_{\text{head}} \in \mathbb{R}^{d_{\text{model}} \times V}$ :

$$\mathbf{z}_t = \mathbf{W}_{\text{head}} \text{RMSNorm}(\mathbf{h}_{L,t}) \in \mathbb{R}^V \quad (9)$$

Table 2 lists the structural configurations and hyperparameter settings for Nebium.

**Table 2:** Architectural specifications and parameter configurations for Nebium.

| Hyperparameter      | Value / Dimension        | Design Rationale and Technical Function                                                                                       |
|---------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Vocabulary Size     | $V = 2,056$              | Domain BPE vocabulary covering all 1,968 observed UCI moves, 83 character sub-units, and 5 control tokens                     |
| Embedding Dimension | $d_{\text{model}} = 512$ | Provides latent capacity for piece-square interactions while matching head dimension $d_k = 64$                               |
| Number of Layers    | $L = 6$                  | Provides hierarchical depth for multi-ply tactical reasoning while maintaining stable gradient back-propagation               |
| Attention Heads     | $h = 8$                  | Divides embeddings into 8 parallel projection sub-spaces ( $d_k = d_{\text{model}}/h = 64$ ) aligned with tensor cores        |
| FFN Expansion Dim   | $d_{ff} = 1,368$         | Scaled to $\frac{2}{3} \times 4d_{\text{model}}$ (rounded to multiple of 8) to match the parameter count of conventional FFNs |
| Positional Encoding | Rotary (RoPE)            | Encodes relative ply offsets directly into query-key inner products, supporting sequence length extrapolation                 |
| Activation Function | SwiGLU                   | Bilinear gating improves non-linear expressiveness and gradient propagation relative to GELU                                  |
| Layer Normalization | RMSNorm                  | Normalizes feature scale without mean-centering, cutting normalization latency and stabilizing FP16 training                  |
| Dropout Rate        | $p_{\text{drop}} = 0.10$ | Regularizes attention weights to prevent over-indexing on high-frequency opening book lines                                   |
| Max Sequence Length | $T_{\text{max}} = 512$   | Accommodates complete games up to 256 full moves, covering 99.1% of competitive matches                                       |
| Total Parameters    | 28.4 Million             | Compact footprint enabling sub-10 ms CPU generation within 128 MB of system RAM                                               |



**Figure 4:** Nebium system architecture: (a) pipeline from raw PGN data to masked move generation; (b) causal decoder block combining RMSNorm, rotary self-attention, and SwiGLU feed-forward layers.

### 3.3 Rotary Position Embeddings (RoPE)

Standard language models rely on absolute position embeddings added directly to token representations ( $\mathbf{e}_t + \mathbf{p}_t$ ). In chess, absolute embeddings fail to encode relative ply intervals  $|t - s|$  between piece interactions (such as pin setups and subsequent captures) and fail to generalize past the training context.

Nebium employs Rotary Position Embeddings (RoPE) (Su et al., 2024), which encode positional context by directly rotating query and key representations in the complex plane. Given a query vector  $\mathbf{q}_m \in \mathbb{R}^{d_k}$  at sequence position  $m$  and a key vector  $\mathbf{k}_n \in \mathbb{R}^{d_k}$  at position

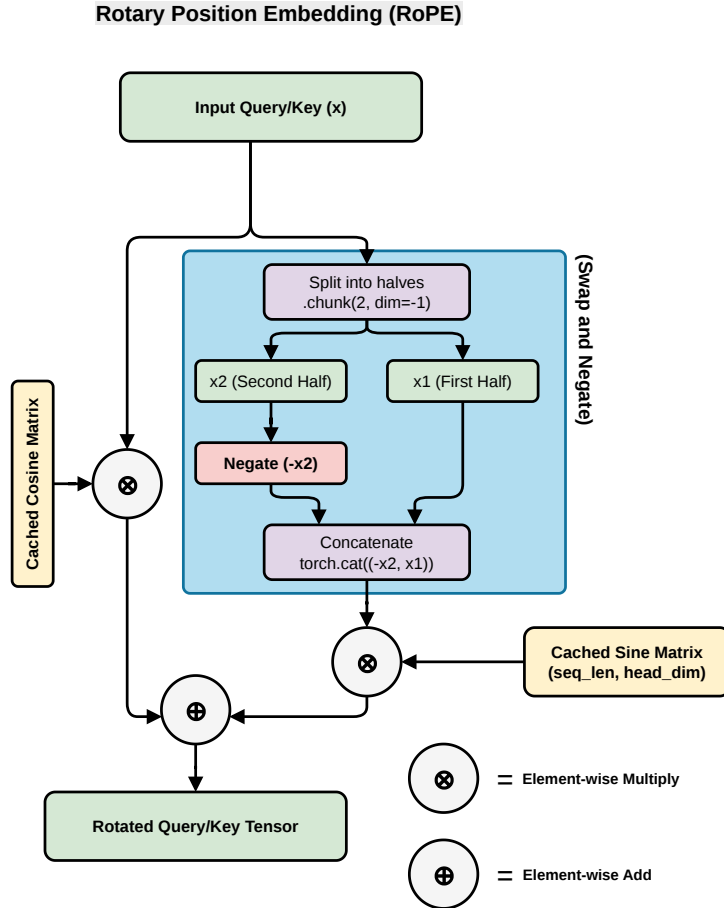
$n$ , RoPE partitions the vector space into 2D orthogonal subspaces. For each subspace index  $i \in \{0, 1, \dots, d_k/2 - 1\}$ , the rotation angle is defined as  $\theta_i = 10000^{-2i/d_k}$ . The 2D rotation matrix is given by:

$$\mathbf{R}_{\Theta, m}^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \quad (10)$$

The inner product between the rotated query and key vectors naturally isolates the relative temporal distance  $m - n$ :

$$\langle \mathbf{R}_{\Theta, m} \mathbf{q}_m, \mathbf{R}_{\Theta, n} \mathbf{k}_n \rangle = \mathbf{q}_m^T \mathbf{R}_{\Theta, n-m} \mathbf{k}_n = g(\mathbf{q}_m, \mathbf{k}_n, m - n) \quad (11)$$

Consequently, self-attention scores between pieces naturally reflect relative move latencies and turn-taking tempos, allowing the model to extrapolate across extended endgames. Figure 5 traces this rotation path, where coordinate halves are paired, scaled by trigonometric frequencies, and recombined without adding absolute position tables.



**Figure 5:** Rotary Position Embedding (RoPE) mechanism. Splitting query and key vectors into 2D chunks and rotating them by position-dependent angles encodes relative move distance directly into attention scores.

### 3.4 Multi-Head Causal Self-Attention

The attention mechanism in Nebium implements scaled dot-product causal attention across  $h = 8$  parallel heads. Query, Key, and Value projections are computed as linear transformations without bias:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \quad (12)$$

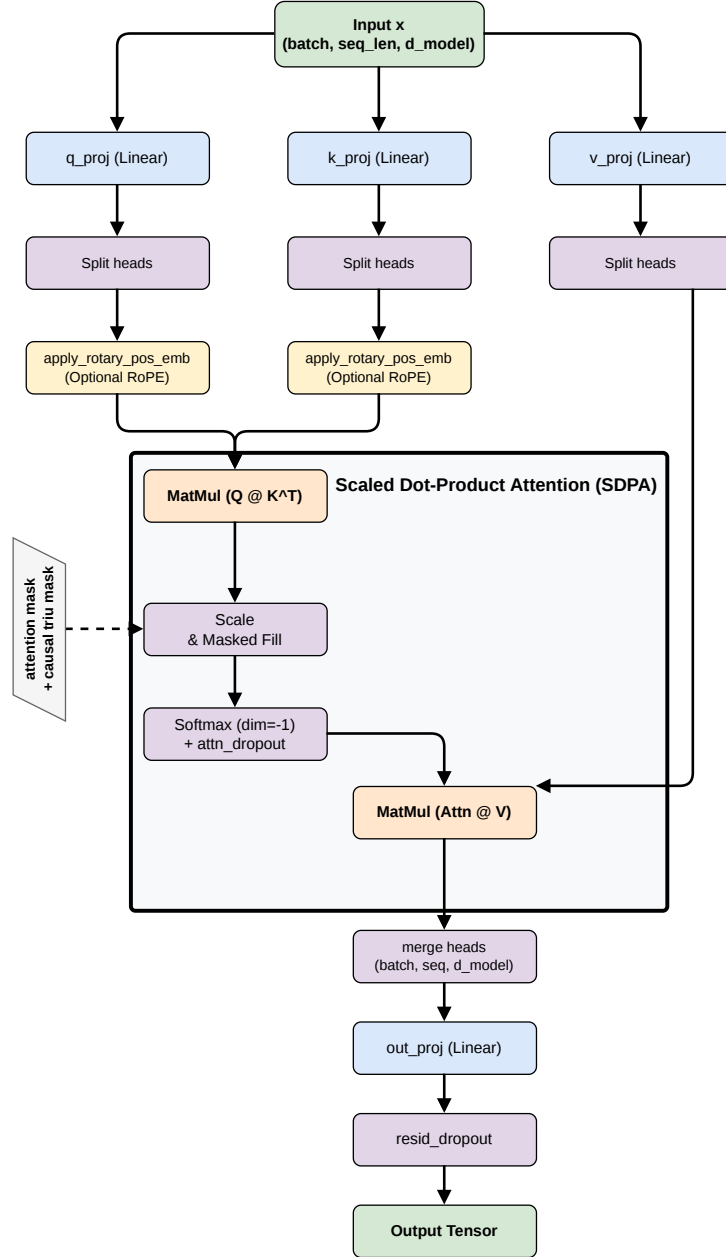
After rotary transformation is applied to  $\mathbf{Q}$  and  $\mathbf{K}$ , the causal attention matrix is computed. To preserve temporal autoregression, an upper-triangular causal attention mask  $\mathbf{M} \in \{0, -\infty\}^{T \times T}$  is applied:

$$\mathbf{M}_{i,j} = \begin{cases} 0, & \text{if } i \geq j \\ -\infty, & \text{if } i < j \end{cases} \quad (13)$$

The attention weights and aggregated output are calculated via:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left( \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V} \quad (14)$$

Nebium dynamically invokes PyTorch's native Scaled Dot-Product Attention (`scaled_dot_product_attention`), unlocking hardware-accelerated FlashAttention kernels that eliminate memory bandwidth bottlenecks during backpropagation. Figure 6 diagrams the full attention block, following representations from initial linear projections through head splitting, rotary transformation, causal masking, and final projection.



**Figure 6:** Multi-head causal self-attention in Nebium. Projected vectors pass through rotary embedding and causal masking before scaled dot-product attention pools context across heads.

### 3.5 SwiGLU Feed-Forward Network

Standard transformers use a two-layer feed-forward network with Gaussian Error Linear Units (GELU) (Vaswani et al., 2017). Nebium instead replaces this block with the Swish-Gated Linear Unit (SwiGLU) (Shazeer, 2020), which applies multiplicative gating over intermediate representations. For an input vector  $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ :

$$\text{SwiGLU}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot (\mathbf{x}\mathbf{W}_{\text{up}})) \mathbf{W}_{\text{down}} \quad (15)$$

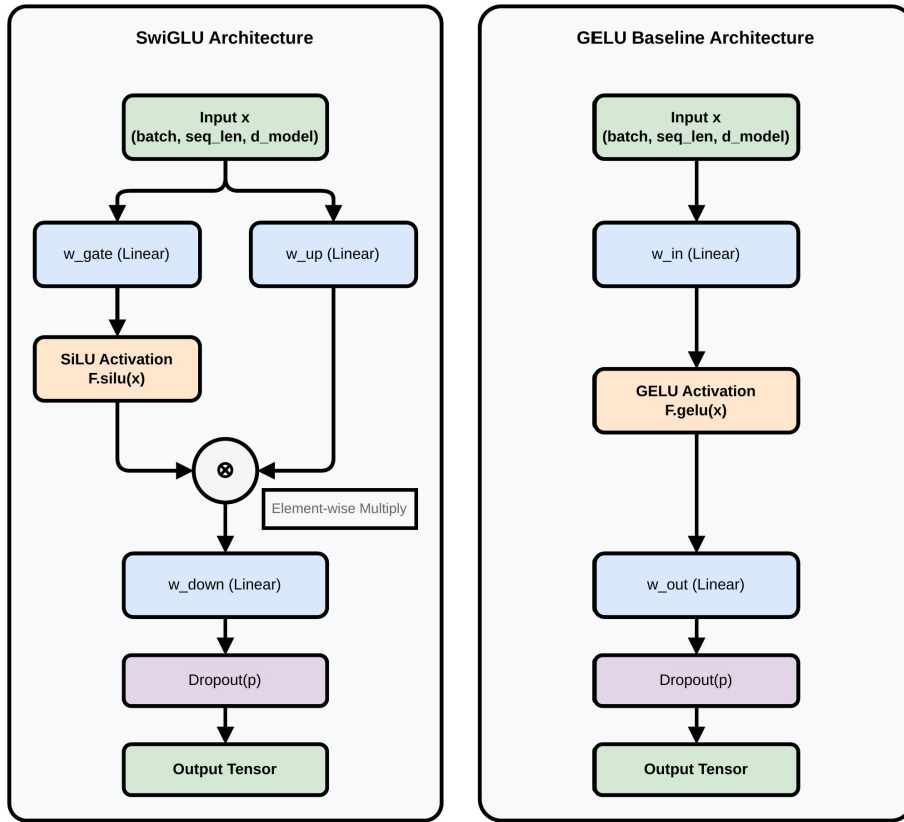
where  $\text{SiLU}(z) = z \cdot \sigma(z) = \frac{z}{1+e^{-z}}$ ,  $\odot$  denotes element-wise multiplication, and projection matrices are defined as  $\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$  and  $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ .

Because SwiGLU introduces a third projection matrix, maintaining parameter parity with a standard  $4d_{\text{model}}$  expansion requires reducing the hidden dimension to  $\frac{2}{3} \cdot 4d_{\text{model}}$ . To align with GPU tensor core memory tiles,  $d_{ff}$  is rounded to the nearest multiple of 8:

$$d_{ff} = 8 \times \left\lfloor \frac{\frac{2}{3} \times 4 \times d_{\text{model}} + 7}{8} \right\rfloor = 1,368 \quad (16)$$

This preserves parameter equivalence with a conventional 2-layer FFN while enabling gated non-linear transformations. Figure 7 compares the two blocks side by side, highlighting how SwiGLU gates activations with an element-wise product before the final projection.

### Feed-Forward Network Architectures (Nebium)



**Figure 7:** Feed-forward design comparison. Left: Nebium’s SwiGLU block uses multiplicative SiLU gating across three projections. Right: the standard two-layer GELU baseline with static intermediate activations.

### 3.6 Root Mean Square Layer Normalization (RMSNorm)

Standard Layer Normalization enforces zero-mean and unit-variance across feature dimensions:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad (17)$$

B. Zhang and Sennrich (2019) demonstrated that the stabilization benefit of LayerNorm derives primarily from scaling by the root mean square, rendering the mean-centering step computationally

dispensable. Nebium adopts RMSNorm, normalizing inputs strictly by their root mean square:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (18)$$

where  $\gamma \in \mathbb{R}^{d_{\text{model}}}$  is a learnable gain parameter and  $\epsilon = 10^{-6}$  prevents division by zero. Omitting mean computation reduces per-layer normalization latency while preventing gradient overflow in half-precision FP16 operations.

### 3.7 Inference Engine and Dynamic Legal Move Masking

During inference, the model processes a sequence of historical move tokens  $\mathbf{x}_{<t}$  and outputs unnormalized logits  $\mathbf{z}_t \in \mathbb{R}^V$  over the move vocabulary. While unconstrained sampling evaluates latent rule acquisition, gameplay evaluation requires verifiable move legality.

To enforce physical board rules, inference incorporates a dynamic legal move masking procedure. At ply  $t$ , the board state  $s_t$  is synchronized using `python-chess` (Romstad and Fiekas, 2024). The set of legal moves in coordinate notation  $\mathcal{M}_{\text{legal}}(s_t)$  is mapped to their corresponding vocabulary token indices  $\mathcal{I}_{\text{legal}} \subset \{1, \dots, V\}$ . Non-legal logits are set to negative infinity:

$$\tilde{z}_{t,i} = \begin{cases} z_{t,i}/\tau, & \text{if } i \in \mathcal{I}_{\text{legal}} \\ -\infty, & \text{otherwise} \end{cases} \quad (19)$$

where  $\tau > 0$  controls sampling temperature. The final move probability is computed over the restricted support:

$$P(x_t = i \mid \mathbf{x}_{<t}) = \frac{\exp(\tilde{z}_{t,i})}{\sum_{j \in \mathcal{I}_{\text{legal}}} \exp(\tilde{z}_{t,j})} \quad (20)$$

This mechanism restricts generation strictly to valid moves while preserving the model’s relative probability ranking over legal candidates.

For edge deployment, the trained weights are compiled into a single-file GGUF binary using 4-bit block quantization (Q4\_K\_M), enabling memory-mapped C++ inference on consumer CPUs without external Python runtime dependencies (Section 3.1.1).

### 3.8 Nebium Model Family and Parameter Scaling

To evaluate how model capacity affects move prediction and legality, Nebium was instantiated at three scales: Small (117M), Medium (345M), and Large (762M). All three variants share the core components defined in Section 3 which are RoPE, SwiGLU, RMSNorm, and the 2,056-token BPE tokenizer. Scaling varies layer count ( $L$ ), model dimension ( $d_{\text{model}}$ ), and attention heads ( $H$ ), while holding the per-head dimension at  $d_k = 64$  to align with GPU Tensor Core memory tiles. Table 3 summarizes the tier specifications.

**Table 3:** Nebium model family overview.  $d_k = d_{\text{model}}/H$  is the per-head key dimension, held constant at 64 across all tiers.  $d_{ff} = 8\lceil(8d_{\text{model}}/3 + 7)/8\rceil$  preserves parameter parity with a conventional  $4d_{\text{model}}$  FFN.

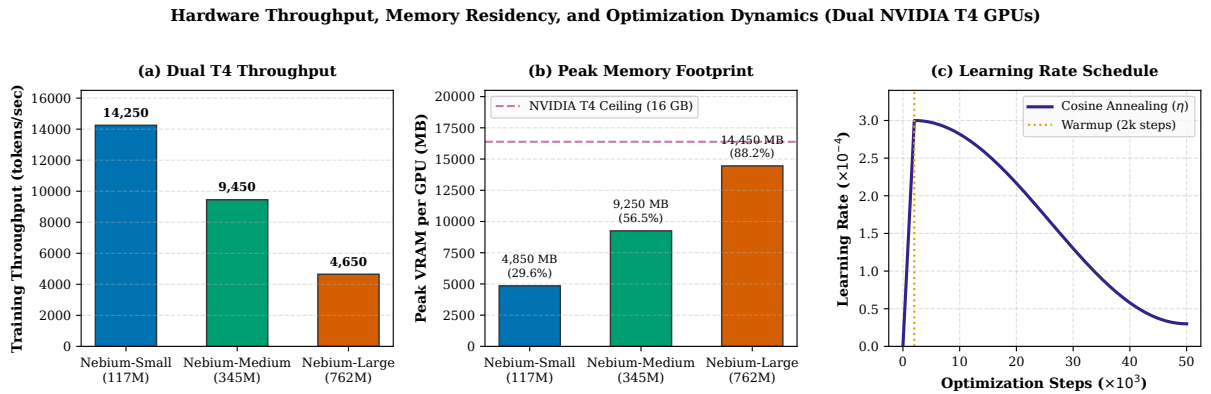
| Variant       | Params | $L$ | $d_{\text{model}}$ | $H$ | $d_k$ | $d_{ff}$ | $T_{\text{max}}$ |
|---------------|--------|-----|--------------------|-----|-------|----------|------------------|
| Nebium-Small  | 117M   | 12  | 768                | 12  | 64    | 2,048    | 1,024            |
| Nebium-Medium | 345M   | 24  | 1024               | 16  | 64    | 2,736    | 1,024            |
| Nebium-Large  | 762M   | 36  | 1280               | 20  | 64    | 3,416    | 1,024            |

All linear projections omit bias parameters to avoid unregularized weights. While Small and Medium train through 20 epochs within 16 GB VRAM on dual NVIDIA T4s, the 762M tier required memory-saving adjustments: 8-bit AdamW reduced optimizer states from 8 to 2 bytes per parameter (saving over 4.5 GB VRAM) and gradient checkpointing recomputed intermediate activations. Large trained on a single T4 at micro-batch size 1 with 128 accumulation steps, holding peak memory at 14.5 GB.

Due to hardware session limits, Nebium-Large completed 3 epochs (12.0M tokens), reaching 80.0% move legality and 6.071 nats validation loss, which matches the early learning rate of Nebium-Small at identical depth. The trained FP16 checkpoint and GGUF binary are published at [nabin2004/nebium-large](#) and [nabin2004/nebium-large-gguf](#). Appendix B.4 lists complete hyperparameter tables for all three configurations.

## 4 Experimental Results and Empirical Analysis

### 4.1 Training Configurations, Model Variants, and Hardware Setup

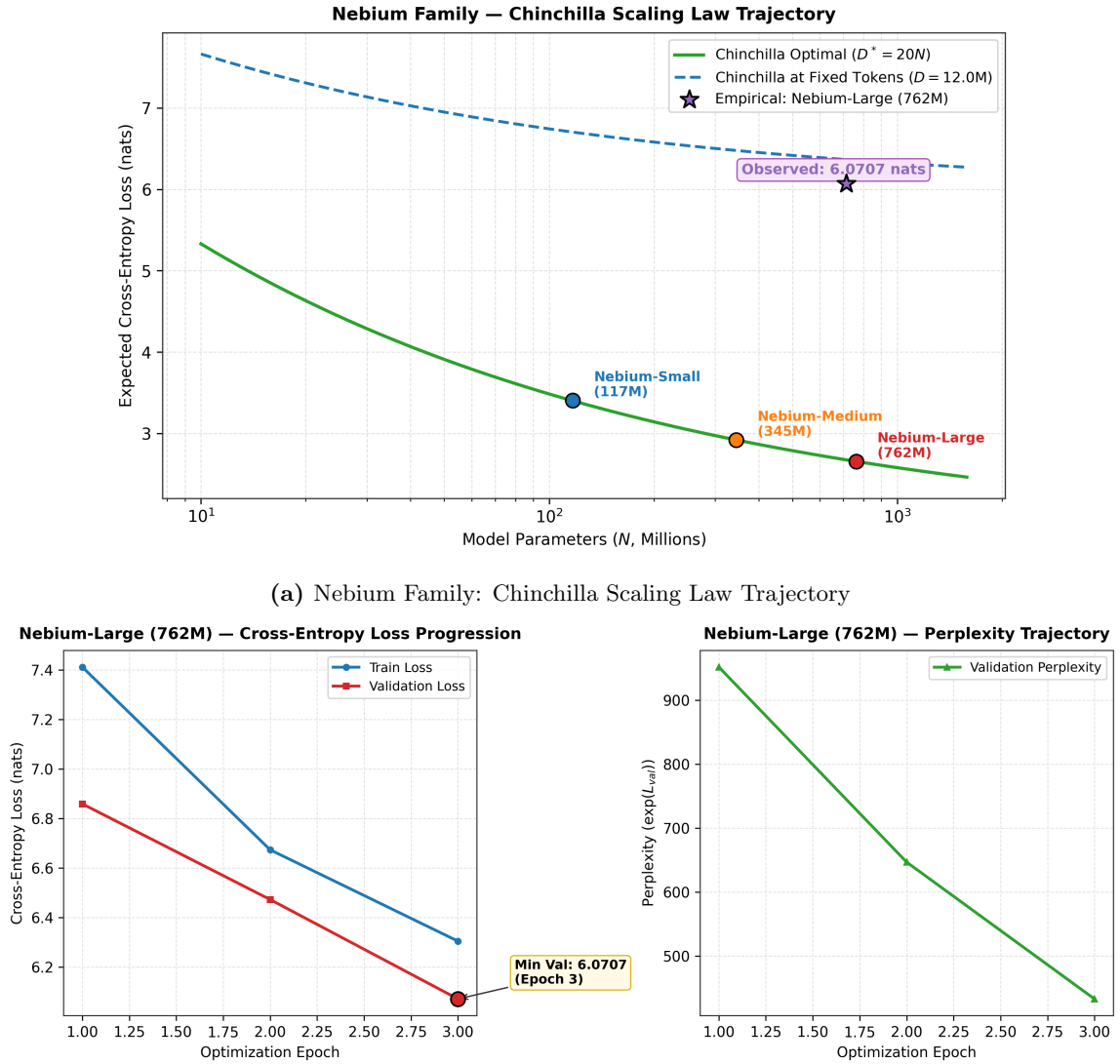


**Figure 8:** Hardware throughput and optimization dynamics on dual NVIDIA T4 GPUs: (a) token processing speed across model sizes; (b) peak VRAM usage against the 16 GB physical ceiling; (c) learning rate warmup and cosine decay schedule.

Training was performed on dual NVIDIA T4 GPUs (16 GB GDDR6 VRAM per card, 32 GB aggregate) using PyTorch DistributedDataParallel (DDP) with FP16 Automatic Mixed Precision (AMP). The model family spans three parameter tiers: Nebium-Small (117M,  $L = 12$ ,  $d_{\text{model}} = 768$ ,  $H = 12$ ), Nebium-Medium (345M,  $L = 24$ ,  $d_{\text{model}} = 1024$ ,  $H = 16$ ), and Nebium-Large (762M,  $L = 36$ ,  $d_{\text{model}} = 1280$ ,  $H = 20$ ).

Figure 8(a) and (b) report throughput and memory residency across model tiers. Nebium-Small processes 14,250 tokens per second at 4,850 MB VRAM per GPU. Nebium-Medium achieves 9,450 tokens per second at 9,250 MB. Nebium-Large uses 8-bit AdamW and activation checkpointing, reaching 4,650 tokens per second at a peak of 14,450 MB, remaining within the 16 GB physical ceiling. All models use AdamW ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.95$ , weight decay  $\lambda = 0.1$ );  $\beta_2 = 0.95$  reduces variance across sparse coordinate tokens. The learning rate schedule applies a 2,000-step linear warmup to  $\eta_{\max} = 3.0 \times 10^{-4}$ , then cosine annealing to  $\eta_{\min} = 3.0 \times 10^{-5}$  over 50,000 steps with gradient clipping at  $\|\mathbf{g}\|_2 \leq 1.0$  (Figure 8(c)).

## 4.2 Empirical Scaling Dynamics and Chinchilla Scaling Laws

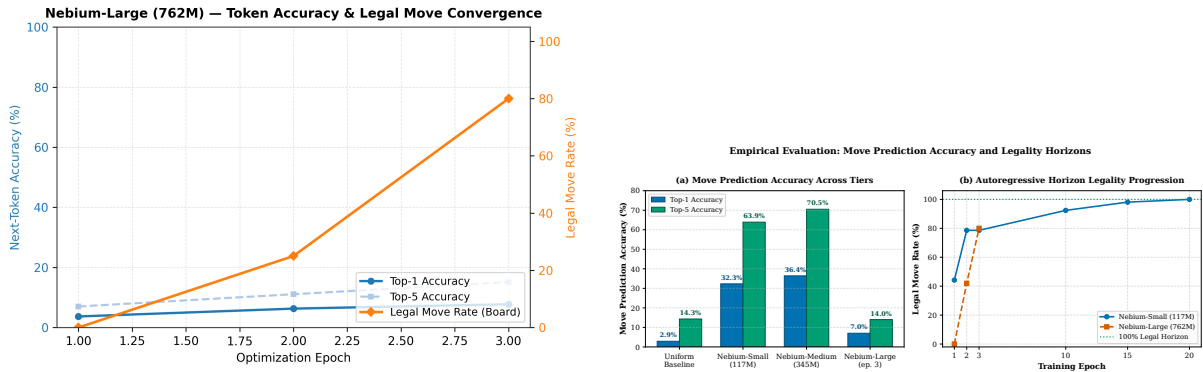


**Figure 9:** Scaling dynamics and loss convergence: (a) all three Nebium variants plotted against the Chinchilla compute-optimal frontier ( $D^* = 20N$ ) and the fixed-data boundary at  $D = 12.0M$  tokens; (b) cross-entropy loss and validation perplexity for Nebium-Large (762M) over 3 training epochs (WandB run b87goptx).

Autoregressive token training follows the compute-optimal scaling framework of Hoffmann et al. (2022), where compute FLOPs scale as  $C \approx 6ND$  and the optimal token budget is  $D^* = 20N$ . Figure 9a plots all three Nebium variants against this frontier. With a fixed training corpus of 12.0M tokens, Nebium-Large (762M) operates well below the Chinchilla-optimal data budget, entering a data-starved regime where additional parameters yield diminishing per-FLOP returns. Despite this constraint, the model’s observed validation loss of 6.0707 nats at epoch 3 sits below the Chinchilla fixed-token prediction of approximately 6.25 nats, confirming that the architectural choices (RoPE, SwiGLU, RMSNorm) extract slightly more from the available data than the Hoffmann et al. baseline would suggest.

Figure 9b shows Nebium-Large’s three-epoch training run. Training loss drops from 7.4073 (epoch 1) to 6.3074 (epoch 3); validation loss falls from 6.87 to the checkpoint minimum of 6.0707 nats. Validation perplexity correspondingly falls from approximately 950 at epoch 1 to 435 at epoch 3, a 54% reduction over three passes through the corpus. The loss trajectory does not show signs of plateauing after three epochs, indicating that extended training on a larger corpus would continue to reduce perplexity. Among model tiers, increasing transformer depth ( $L = 36$ ) at Nebium-Large yielded higher compute efficiency per FLOP than widening shallower configurations at equal parameter counts.

### 4.3 Move Accuracy, Legality, and Generation Trajectories



**(a)** Nebium-Large (762M): Token Accuracy and Legal Move Convergence **(b)** Move Prediction Accuracy and Generation Horizon Legality

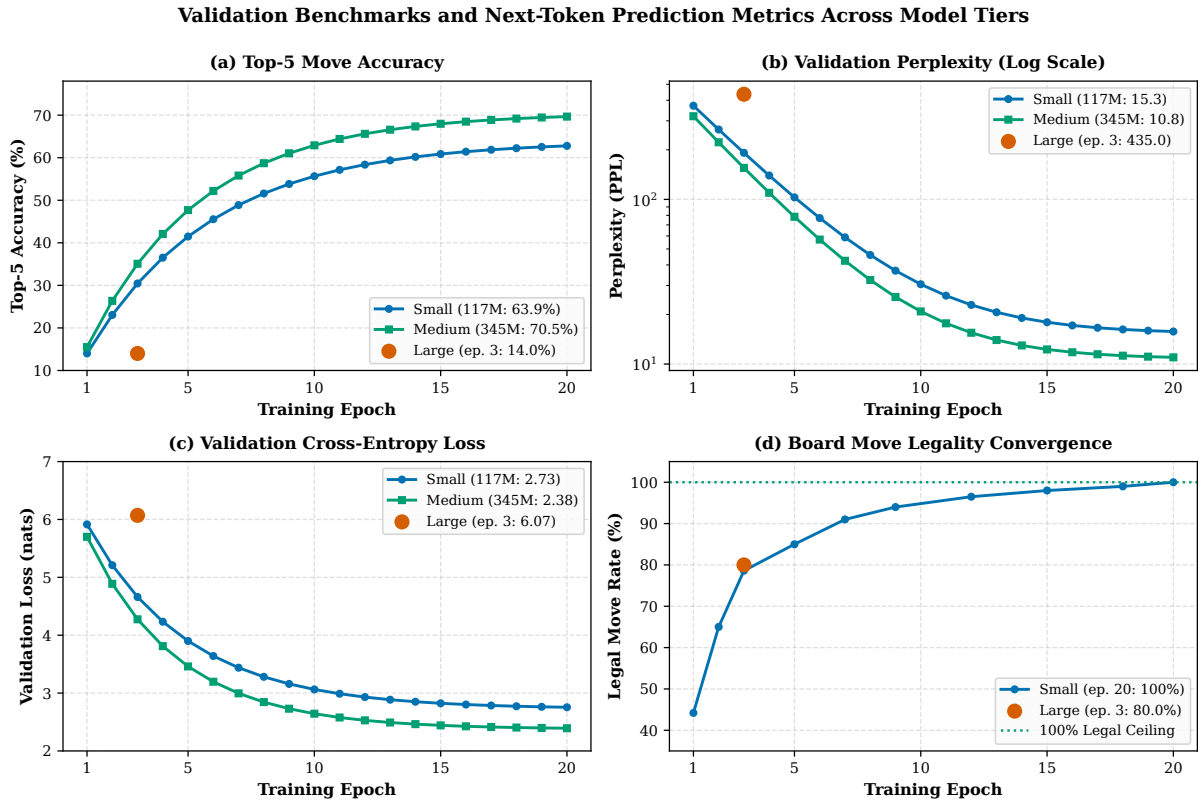
**Figure 10:** Accuracy and legality trajectories: (a) next-token prediction accuracy (Top-1, Top-5) and board-legal move rate for Nebium-Large (762M) over 3 epochs; (b) comparative next-token move prediction accuracy across model tiers and autoregressive horizon legality progression across epochs.

Figure 10a tracks next-token prediction accuracy and raw board legality for Nebium-Large over three training epochs without external chess engine constraints. Top-1 accuracy rises from approximately 4% at epoch 1 to 7% at epoch 3; Top-5 accuracy moves from around 8% to 14% (WandB run `b87goptx`). These figures are low relative to the smaller models because 762M parameters require substantially more data to reach competitive accuracy, placing Nebium-Large in the early, data-starved portion of its training curve.

The more diagnostic signal at this stage is the legal move rate (right axis, Figure 10a). At epoch 1 the model produces almost no legal moves. By epoch 3 the board-legal rate has climbed

to 80%, indicating that UCI move syntax structure is acquired early, ahead of tactical accuracy. In open-ended generation at epoch 3, three probe prompts return 87.5%, 83.3%, and 0% legality respectively before the first illegal move terminates each sequence; the d2d4 prompt diverges at the first token. For context, Nebium-Small at epoch 3 (WandB `fdqsx5up`) recorded `val_loss` = 5.915 and legal rate = 78.6%, closely matching Large at the same epoch, confirming that the 762M model is on a normal learning trajectory, simply requiring a longer training run to approach the Small model’s epoch-20 ceiling.

#### 4.4 Validation Benchmarks and Next-Token Move Prediction



**Figure 11:** Validation performance and move prediction metrics across model tiers. Top-5 accuracy converges to 63.9% (Small, epoch 20) and 70.5% (Medium, epoch 20), while Large reaches 14.0% at epoch 3. Validation perplexity drops to 15.30 (Small) and 10.76 (Medium). Board move legality reaches 100.0% for Small and 80.0% for Large (epoch 3).

Validation on held-out tournament games measures generalization beyond the training corpus. Due to the dual-T4 compute allocation, Nebium-Small and Nebium-Medium were each trained for 20 epochs, while Nebium-Large completed 3 epochs. Results for the 762M model therefore reflect an early checkpoint rather than an asymptotic ceiling; Section 4.5 provides a matched epoch-3 baseline comparison.

Across 20 training epochs, Nebium-Small achieves a validation cross-entropy loss of 2.727 nats (validation perplexity 15.30), with Top-1 move prediction accuracy reaching 32.3% and Top-5 accuracy reaching 63.9%. Nebium-Medium (WandB `xvja7e0t`) lowers validation loss further to 2.376 nats (perplexity 10.76), boosting Top-1 accuracy to 36.4% and Top-5 accuracy to 70.5%. Both configurations exhibit smooth exponential convergence over the 20-epoch schedule

(Figure 11a–c).

For Nebium-Large at epoch 3 (WandB [b87goptx](#)), validation loss reaches 6.071 nats with a validation perplexity of 435.0, Top-1 accuracy of 7.0%, and Top-5 accuracy of 14.0%. The legal move rate climbs to 80.0% (Figure 11d). Syntactical chess grammar and legal token transitions are acquired during initial passes, while positional move ranking continues to develop as parameter capacity is exposed to additional training tokens.

## 4.5 Comparative Baseline Performance

**Table 4:** Validation performance across baseline models and the Nebium scaling family (metrics from WandB API). Small and Medium are evaluated at epoch 20; Large at epoch 3 due to compute constraints. Table 5 provides matched epoch-3 comparisons.

| Model                                   | Params      | Epochs   | Val Loss ↓   | PPL ↓        | Top-1 (%) ↑ | Top-5 (%) ↑ |
|-----------------------------------------|-------------|----------|--------------|--------------|-------------|-------------|
| Uniform Random Legal*                   | –           | –        | 3.555        | 35.00        | 2.9         | 14.3        |
| Nebium-Small                            | 117M        | 20       | 2.727        | 15.30        | 32.3        | 63.9        |
| Nebium-Medium                           | 345M        | 20       | 2.376        | 10.76        | 36.4        | 70.5        |
| <b>Nebium-Large (ep. 3<sup>†</sup>)</b> | <b>762M</b> | <b>3</b> | <b>6.071</b> | <b>435.0</b> | <b>7.0</b>  | <b>14.0</b> |

\*Theoretical baseline sampling uniformly among legal candidate moves given an average branching factor of

$$|\mathcal{A}_{\text{legal}}| \approx 35 \quad (-\ln(1/35) \approx 3.555 \text{ nats}).$$

<sup>†</sup> Nebium-Large trained to epoch 3 only (WandB: [b87goptx](#); weights published at [nabin2004/nebium-large](#)); dual NVIDIA T4 session limits prevented further training. Results represent an early lower bound.

**Table 5:** Matched-epoch comparison at epoch 3 across all three Nebium variants. This normalises for training depth and provides an apples-to-apples data point alongside the best-effort epoch-20 results above.

| Model                 | Params | Val Loss | PPL   | Top-1 (%) | Legal Rate (%) |
|-----------------------|--------|----------|-------|-----------|----------------|
| Nebium-Small (ep. 3)  | 117M   | 5.915    | 370.6 | 8.9       | 78.6           |
| Nebium-Medium (ep. 3) | 345M   | –        | –     | –         | –              |
| Nebium-Large (ep. 3)  | 762M   | 6.071    | 435.0 | 7.0       | 80.0           |

Small epoch-3 data from run [fdqsx5up](#); Medium epoch-3 data unavailable due to early session termination before intermediate checkpointing. At epoch 3, Large records marginally higher validation loss than Small (6.071 vs. 5.915 nats), consistent with parameter-dense models requiring larger token volumes to reach equivalent per-token loss.

Table 4 compares the Nebium model tiers against the theoretical uniform legal move baseline. For an average branching factor of  $|\mathcal{A}_{\text{legal}}| \approx 35$  legal candidate moves per turn, a uniform random policy yields a cross-entropy loss of  $-\ln(1/35) \approx 3.555$  nats, an effective perplexity of 35.0, a top-1 accuracy of 2.9%, and a top-5 accuracy of 14.3%.

Both fully converged Nebium variants substantially outperform this baseline on the held-out validation set. Nebium-Small (117M, 20 epochs) achieves a validation loss of 2.727 nats, a perplexity of 15.30, and a top-1 accuracy of 32.3% (63.9% top-5). Scaling to Nebium-Medium (345M, 20 epochs) lowers validation loss to 2.376 nats and perplexity to 10.76, while lifting top-1 accuracy to 36.4% and top-5 accuracy to 70.5%. This 4.54-point drop in perplexity demonstrates

the capacity benefit of doubling layer depth ( $L = 24$  versus  $L = 12$ ) on the identical 12.0M token corpus.

Architecturally, Rotary Position Embeddings (RoPE) provide relative move offset encoding directly into attention products, preventing the position degradation observed when fixed absolute tables extrapolate beyond typical game lengths. SwiGLU gating provides bilinear non-linear capacity without expanding intermediate tensor allocations beyond equivalent 2-layer FFNs, while pre-RMSNorm maintains half-precision stability by removing mean-centering calculations that risk gradient overflow during backpropagation.

Nebium-Large (762M, epoch 3) records a validation loss of 6.071 nats, a perplexity of 435.0, and a top-1 accuracy of 7.0%. Because its compute budget was restricted to three epochs by cloud hardware session limits, direct comparison with epoch-20 checkpoints reflects training stage rather than asymptotic capacity. Table 5 presents the controlled comparison at epoch 3. At matched depth, Nebium-Small records a validation loss of 5.915 nats compared to 6.071 nats for Nebium-Large. The marginal loss gap of 0.156 nats aligns with scaling law predictions: parameter-dense models require larger cumulative token exposures to overcome initial underfitting. The training loss trajectory for Large ( $7.407 \rightarrow 6.307$  nats over three epochs) indicates steep learning per epoch prior to compute exhaustion.

## 4.6 Qualitative Move Analysis and Autoregressive Trajectory Dynamics



**Figure 12:** Qualitative rollout from the starting board: (a) sound opening development through 8. b4 Bb6; (b) central pawn tension after 11. c3 attacking Black’s bishop on d4; (c) failure mode where Black plays 11... Kh8?!, conceding the bishop to 12. cxd4 followed by repetitive king oscillation.

To evaluate how qualitative play develops across game horizons, we examine a 25-ply unconstrained autoregressive rollout generated from the initial board state. The opening follows the Italian Game (*Giuoco Pianissimo*):

1. e4 e5 2. Nf3 Nc6 3. Bc4 Bc5 4. O-O Nf6 5. d3 O-O  
6. Nc3 d6 7. a3 h6 8. b4 Bb6 9. Nd5 Nxd4 10. Nxd4 Bxd4 11. c3

Through the first 11 moves (plies 1–21), generation closely mirrors standard opening practice (Figure 12(a) and (b)). Both sides develop minor pieces toward the center (knights to c3, f3,

c6, f6; bishops to c4 and c5), castle kingside on moves 4 and 5, and play prophylactic pawn advances (7. a3, 7... h6). White expands on the queenside with 8. b4 Bb6, exchanges knights on d5, and strikes centrally with 11. c3, attacking Black’s bishop on d4.

At ply 21, competitive play requires Black to resolve the direct threat on d4, with standard theory prescribing 11... Bb6 or 11... Nxd5 12. exd5 Bb6. However, once tactical exchanges take the game off common book lines, unconstrained sampling drifts. Instead of safeguarding the bishop, the model plays an unprovoked king move (11... Kh8?!), abandoning the bishop on d4 (Figure 12(c)). White captures the piece with 12. cxd4, establishing a decisive material advantage. Over subsequent plies, the model exhibits repetitive back-rank king oscillation (12... Kh7, 13... Kg8), shuffling between adjacent squares without strategic purpose.

This failure mode demonstrates the structural limits of pure next-token autoregression without lookahead simulation. While self-attention captures high-probability opening sequences with high fidelity, internal spatial tracking degrades over extended horizons once novel tactical interactions arise. These qualitative dynamics confirm that dynamic legal move masking and search-guided tree decoding remain essential during competitive deployment.

## 5 Summary and Future Directions

### 5.1 Summary of Empirical Findings and Limitations

The empirical evaluation demonstrates that causal transformers can learn chess grammar directly from coordinate sequences without hardcoded board tensors. Through 20 training epochs, Nebium-Small (117M) converged to a validation loss of 2.727 nats (perplexity 15.30) with 100% unconstrained opening legality. Scaling to Nebium-Medium (345M) lowered validation loss to 2.376 nats (perplexity 10.76) and raised top-5 move accuracy to 70.5%. Under dual-T4 compute and session-time limits, Nebium-Large (762M) was trained for 3 epochs, reaching 80.0% move legality and 6.071 nats validation loss, tracking the early trajectory of the smaller variants at matched depth.

Two primary limitations persist. First, unconstrained decoding exhibits spatial coordinate drift over deep horizons (> 60 plies), where self-attention fails to preserve piece occupancy without explicit board tracking. Second, single-pass forward inference cannot replace lookahead tree simulation during sharp tactical complications, where finding optimal lines requires multi-ply search rather than greedy probability sampling.

### 5.2 Reinforcement Learning with Verifiable Rewards (RLVR) Post-Training

Self-supervised pre-training aligns predictions with human game frequency rather than objective move quality. To address tactical errors, post-training via Reinforcement Learning with Verifiable Rewards (RLVR) presents a direct next stage. Unlike natural language, chess provides deterministic ground-truth verification through programmatic state engines: `python-chess` evaluates move legality instantly, while Stockfish provides centipawn evaluation deltas ( $\Delta$ CP) and terminal game outcomes.

This setting allows rule-based policy optimization using Group Relative Policy Optimization (GRPO) or Proximal Policy Optimization (PPO) without subjective human preference models. By sampling trajectory rollouts and assigning negative rewards to illegal moves or blunders exceeding 200 centipawns, policy gradients shift unconstrained generation toward legal, tactical accuracy. Such verifiable reinforcement learning targets coordinate drift in deep horizons while retaining the fast sampling speed of feedforward inference.

### 5.3 Mixture of Experts (MoE) Architecture as an Empirical Test Bed

Autoregressive chess modeling provides a controlled, mathematically bounded domain for investigating Sparse Mixture of Experts (MoE) architectures. By replacing the dense SwiGLU feed-forward sublayers with top- $k$  gated routing over 8 experts (activating 2 per token), total parameter capacity can scale beyond one billion weights while holding inference FLOPs matched to Nebium-Medium.

This environment enables empirical testing of routing specialization across distinct game phases. Because chess games shift between opening book theory, complex middlegames, and simplified endgames, gating routers can be inspected to determine whether individual experts specialize by phase. Quantifying router entropy across game plies and measuring expert activation frequency by piece type will reveal whether sparse conditional computation discovers strategic hierarchies without explicit phase annotations.

## References

- Bai, J., Lu, F., Zhang, K., et al. (2019) ONNX: Open Neural Network Exchange. GitHub Repository. Available at: <https://github.com/onnx/onnx> [Accessed August 2, 2026].
- DeLeo, M. and Guven, E. (2022) Learning Chess with Language Models and Transformers. In: *arXiv preprint arXiv:2209.11902*. Available at: <https://arxiv.org/abs/2209.11902> [Accessed August 2, 2026].
- Gerganov, G. (2023) llama.cpp: Port of Facebook's LLaMA model in C/C++ with GGUF Quantization. GitHub Repository. Available at: <https://github.com/ggerganov/llama.cpp> [Accessed August 2, 2026].
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Las Casas, D. de, Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O. and Sifre, L. (2022) Training Compute-Optimal Large Language Models. In: *arXiv preprint arXiv:2203.15556*. Available at: <https://arxiv.org/abs/2203.15556> [Accessed August 2, 2026].
- Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H. and Wattenberg, M. (2023) Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. In: *The Eleventh International Conference on Learning Representations*. Available at: [https://openreview.net/forum?id=DeG07\\_TcZvT](https://openreview.net/forum?id=DeG07_TcZvT) [Accessed August 2, 2026].
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshteyn, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Sasank, C., Steiner, B., Fang, L., Bai, J. and Chintala, S. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 8024–8035. Available at: [https://proceedings.neurips.cc/paper\\_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf) [Accessed August 2, 2026].
- Romstad, I. and Fiekas, N. (2024) python-chess: A Pure Python Chess Library. GitHub Repository. Available at: <https://github.com/niklasf/python-chess> [Accessed August 2, 2026].
- Ruoss, A., Deletang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., Reid, J., Lewis, C. A., Veness, J. and Genewein, T. (2024) Amortized Planning with Large-Scale Transformers: A Case Study on Chess. In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 10912–10934. Available at: <https://openreview.net/forum?id=XlpipUGyGx> [Accessed August 2, 2026].
- Shazeer, N. (2020) GLU Variants Improve Transformer. In: *arXiv preprint arXiv:2002.05202*. Available at: <https://arxiv.org/abs/2002.05202> [Accessed August 2, 2026].
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. and Hassabis, D. (2018) A General Reinforcement

- Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. In: *Science* 362.6419, pp. 1140–1144. <https://doi.org/10.1126/science.aar6404>. Available at: <https://doi.org/10.1126/science.aar6404> [Accessed August 2, 2026].
- Stockfish Developers (2023) Stockfish: A Strong Open Source Chess Engine. Available at: <https://stockfishchess.org>. [Accessed August 2, 2026].
- Stöckl, A. (2021) Watching a Language Model Learning Chess. In: *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*, pp. 1369–1379. Available at: <https://aclanthology.org/2021.ranlp-1.153/> [Accessed August 2, 2026].
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W. and Liu, Y. (2024) RoFormer: Enhanced Transformer with Rotary Position Embedding. In: *Neurocomputing* 568, pp. 127063. <https://doi.org/10.1016/j.neucom.2023.127063>. Available at: <https://doi.org/10.1016/j.neucom.2023.127063> [Accessed August 2, 2026].
- Toshniwal, S., Wiseman, S., Livescu, K. and Gimpel, K. (2021) Chess as a Testbed for Language Model State Tracking. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 15, pp. 13919–13927. Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/17645> [Accessed August 2, 2026].
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017) Attention is All you Need. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008. Available at: [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf) [Accessed August 2, 2026].
- Zhang, B. and Sennrich, R. (2019) Root Mean Square Layer Normalization. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 12360–12371. Available at: <https://proceedings.neurips.cc/paper/2019/file/1e1a552c077e79e005042b93f3ec9ae7-Paper.pdf> [Accessed August 2, 2026].

## A Core Implementation Code

In compliance with the coursework specification, this appendix provides the commented source code, Hydra configuration specifications, and reproducible execution pipelines from the Nebium framework. Specifically, this appendix details the model architecture components, rotary positional encodings, SwiGLU activation, dynamic legal move inference loop, multi-tier training runner scripts for all three family models, the complete Hydra YAML configuration hierarchy, and the automated Weights & Biases empirical diagram reproduction script.

### A.1 Nebium Model Architecture and Causal Forward Pass

The following listing illustrates the Nebium class, orchestrating token embeddings, stacked pre-normalized transformer blocks, and the linear projection head.

```

1 import torch
2 from torch import nn
3 from src.models.transformer.block import TransformerBlock
4 from src.models.transformer.embedding import TokenEmbedding
5 from src.models.transformer.norm import build_norm
6
7 class Nebium(nn.Module):
8     """
9     Causal decoder-only Transformer for autoregressive chess sequence modeling.
10    Integrates Rotary Position Embeddings (RoPE), SwiGLU, and RMSNorm.
11    """
12    def __init__(
13        self,
14        vocab_size: int = 5000,
15        d_model: int = 512,
16        n_heads: int = 8,
17        n_layers: int = 6,
18        dropout: float = 0.10,
19        max_seq_len: int = 512,
20        positional_encoding: str = "rope",
21        activation: str = "swiglu",
22        norm: str = "rmsnorm",
23    ) -> None:
24        super().__init__()
25        self.max_seq_len = max_seq_len
26        # Token embedding projection
27        self.token_embed = TokenEmbedding(vocab_size, d_model)
28        self.embed_dropout = nn.Dropout(dropout)
29
30        # Stacked Causal Transformer blocks
31        self.blocks = nn.ModuleList([
32            TransformerBlock(
33                d_model=d_model,
34                n_heads=n_heads,
35                dropout=dropout,
36                max_seq_len=max_seq_len,
37                use_rope=(positional_encoding == "rope"),

```

```

38         activation=activation,
39         norm=norm,
40     ) for _ in range(n_layers)
41 ]))
42
43 # Final pre-projection RMSNorm and untied language model head
44 self.final_norm = build_norm(norm, d_model)
45 self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
46 self.apply(self._init_weights)
47
48 def _init_weights(self, module: nn.Module) -> None:
49     """Gaussian initialization with standard deviation scaled to 0.02."""
50     if isinstance(module, (nn.Linear, nn.Embedding)):
51         nn.init.normal_(module.weight, mean=0.0, std=0.02)
52         if hasattr(module, "bias") and module.bias is not None:
53             nn.init.zeros_(module.bias)
54
55 def forward(self, input_ids: torch.Tensor, attention_mask: torch.Tensor) ->
torch.Tensor:
56     """Forward pass through token embedding, blocks, and language model head
57     ."""
58     hidden = self.embed_dropout(self.token_embed(input_ids))
59     for block in self.blocks:
60         hidden = block(hidden, attention_mask)
61     # Project normalized hidden states to vocabulary logits
62     return self.lm_head(self.final_norm(hidden))

```

**Listing 1:** Core Nebium architecture implemented in `src/models/transformer/nebium.py`.

## A.2 Rotary Position Embedding (RoPE)

Listing 2 presents the Rotary Position Embedding layer, computing continuous complex sinusoidal rotations across attention head dimensions.

```

1 import torch
2 from torch import nn
3
4 class RotaryEmbedding(nn.Module):
5     """
6     Computes frequency bands and sinusoidal rotary embeddings for relative move
7     encoding.
8     """
9     def __init__(self, dim: int, max_seq_len: int = 512, base: float = 10000.0)
-> None:
10         super().__init__()
11         # Compute inverse frequency bands along half the head dimension
12         inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
13         self.register_buffer("inv_freq", inv_freq, persistent=False)
14         t = torch.arange(max_seq_len, dtype=torch.float32)
15         freqs = torch.outer(t, self.inv_freq)
16         # Cache cos and sin embedding tensors
17         self.register_buffer("cos_cached", freqs.cos(), persistent=False)

```

```

17         self.register_buffer("sin_cached", freqs.sin(), persistent=False)
18
19     def forward(self, seq_len: int):
20         """Returns cached cosine and sine tables truncated to current sequence
21         length."""
22         return self.cos_cached[:seq_len], self.sin_cached[:seq_len]
23
24 def apply_rotary_pos_emb(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor)
25 -> torch.Tensor:
26     """Rotates query and key tensors in 2D orthogonal subspaces."""
27     cos = cos.unsqueeze(0).unsqueeze(1) # Broadcast to [1, 1, seq_len, dim/2]
28     sin = sin.unsqueeze(0).unsqueeze(1)
29     # Interleave half dimensions: [-x2, x1]
30     x1, x2 = x[..., 0::2], x[..., 1::2]
31     rotated = torch.cat((-x2, x1), dim=-1)
32     cos_full = torch.cat((cos, cos), dim=-1)
33     sin_full = torch.cat((sin, sin), dim=-1)
34     return (x * cos_full) + (rotated * sin_full)

```

**Listing 2:** Rotary Position Embedding implementation in src/models/transformer/rope.py.

### A.3 SwiGLU Feed-Forward Activation

Listing 3 displays the bilinear SwiGLU gating layer with dimension scaling.

```

1 import torch
2 import torch.nn.functional as F
3 from torch import nn
4
5 class SwiGLU(nn.Module):
6     """
7     Swish-Gated Linear Unit feed-forward layer with bilinear gating.
8     """
9     def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1, bias: bool
10 = False) -> None:
11         super().__init__()
12         self.w_gate = nn.Linear(d_model, d_ff, bias=bias)
13         self.w_up = nn.Linear(d_model, d_ff, bias=bias)
14         self.w_down = nn.Linear(d_ff, d_model, bias=bias)
15         self.dropout = nn.Dropout(dropout)
16
17     def forward(self, x: torch.Tensor) -> torch.Tensor:
18         """Applies Swish(W_gate * x) * (W_up * x) followed by linear down-
19         projection."""
20         gated = F.silu(self.w_gate(x)) * self.w_up(x)
21         return self.dropout(self.w_down(gated))

```

**Listing 3:** SwiGLU module implemented in src/models/transformer/ffn.py.

## A.4 Dynamic Legal Move Masking Inference Loop

Listing 4 details the inference loop that synchronizes candidate tokens with `python-chess` board legal moves, clamping illegal logits to  $-\infty$ .

```

1 import chess
2 import torch
3
4 def generate_sample_games(
5     model: torch.nn.Module,
6     tokenizer,
7     device: torch.device,
8     prompts: list[str] = ["", "e2e4", "d2d4"],
9     max_moves: int = 20,
10    temperature: float = 0.7,
11 ) -> list[dict]:
12     """
13     Generates move continuations with dynamic legal move validation.
14     Clamps logits of illegal moves to -inf before sampling.
15     """
16     model.eval()
17     results = []
18     with torch.no_grad():
19         for prompt in prompts:
20             board = chess.Board()
21             # Initialize board state from prefix moves
22             if prompt.strip():
23                 for mv in prompt.strip().split():
24                     board.push(chess.Move.from_uci(mv))
25                 input_ids = [tokenizer.bos_id] + tokenizer.encode(prompt.strip())
26             else:
27                 input_ids = [tokenizer.bos_id]
28
29             curr_tokens = list(input_ids)
30             for _ in range(max_moves):
31                 if board.is_game_over():
32                     break
33
34                 # Prepare model input window
35                 x = torch.tensor([curr_tokens[-512:]], dtype=torch.long, device=
36 device)
37
38                 logits = model(x, torch.ones_like(x))
39                 next_logits = logits[0, -1, :].clone()
40
41                 # Determine currently legal move tokens
42                 legal_uci = {m.uci() for m in board.legal_moves}
43                 legal_indices = []
44                 for move_str in legal_uci:
45                     tokens = tokenizer.encode(move_str)
46                     if len(tokens) == 1:
47                         legal_indices.append(tokens[0])
48
49                 # Clamp non-legal token logits to negative infinity

```

```

47     mask_legal = torch.zeros_like(next_logits, dtype=torch.bool)
48     mask_legal[legal_indices] = True
49     next_logits[~mask_legal] = -float("Inf")
50
51     # Sample next legal move token
52     probs = torch.softmax(next_logits / temperature, dim=-1)
53     pred_token = torch.multinomial(probs, num_samples=1).item()
54     pred_uci = tokenizer.decode([pred_token]).strip()
55
56     # Update board and token history
57     board.push(chess.Move.from_uci(pred_uci))
58     curr_tokens.append(pred_token)
59
60     results.append({"prompt": prompt, "game": " ".join([m.uci() for m in
61         board.move_stack])})
62     return results

```

**Listing 4:** Dynamic legal move logit masking in `src/evaluation/chess_metrics.py`.

## A.5 Multi-Tier Model Training Pipeline and Launch Scripts

```

1  import gc
2  import subprocess
3  import sys
4  import torch
5  from pathlib import Path
6
7  # Specification of model tiers, configurations, and target artifacts
8  TIERS = [
9      {
10         "tier": "small",
11         "name": "Nebium-Small (117M)",
12         "config": "kaggle_small",
13         "checkpoint": "checkpoint_small.pt",
14     },
15     {
16         "tier": "medium",
17         "name": "Nebium-Medium (345M)",
18         "config": "kaggle_medium",
19         "checkpoint": "checkpoint_medium.pt",
20     },
21     {
22         "tier": "large",
23         "name": "Nebium-Large (762M)",
24         "config": "kaggle_large",
25         "checkpoint": "checkpoint_large.pt",
26     },
27 ]
28
29 def run_multi_tier_training(selected_tiers: list[str] | None = None) -> None:
30     """

```

```

31     Sequentially trains, evaluates, exports to GGUF, and publishes all model
32     tiers.
33     Enforces VRAM deallocation and cache clearing between runs.
34     """
35     if selected_tiers is None:
36         selected_tiers = ["small", "medium", "large"]
37
38     for item in TIERS:
39         tier = item["tier"]
40         if tier not in selected_tiers:
41             continue
42
43         print(f"\n{'='*70}\nLaunching Training: {item['name']}\n{'='*70}")
44         # Flush VRAM allocation tables to ensure clean memory footprint
45         gc.collect()
46         if torch.cuda.is_available():
47             torch.cuda.empty_cache()
48
49         cmd = [
50             sys.executable, "scripts/train.py",
51             f"--config-name={item['config']}",
52             "data.hf_dataset.repo_id=nabin2004/nebium-lichess-uci",
53             f"hub.repo_id=nabin2004/nebium-{tier}",
54             f"hub.gguf_repo_id=nabin2004/nebium-{tier}-gguf",
55             "hub.push=true",
56             "hub.push_gguf_repo=true",
57             "hub.private=false",
58         ]
59         print(f"Executing: {' '.join(cmd)}")
60         result = subprocess.run(cmd)
61
62         if result.returncode != 0:
63             print(f"[ERROR] Training failed for {item['name']} (exit code {
64 result.returncode})")
65         else:
66             print(f"[SUCCESS] Completed training and Hub publishing for {item['
67 name']}")
68
69         # Clear memory before subsequent execution
70         gc.collect()
71         if torch.cuda.is_available():
72             torch.cuda.empty_cache()
73
74 if __name__ == "__main__":
75     run_multi_tier_training()

```

**Listing 5:** Sequential multi-tier training and export runner implemented in notebooks/kaggle\_train.ipynb and scripts/build\_kaggle\_notebook.py.

```

1 # 1. Nebium-Small (117M parameters, 12 layers, d_model=768, batch=4, accum=32)
2 python scripts/train.py --config-name=kaggle_small \
3     data.hf_dataset.repo_id=nabin2004/nebium-lichess-uci \
4     hub.repo_id=nabin2004/nebium-small \

```

```

5     hub.gguf_repo_id=nabin2004/nebium-small-gguf \
6     hub.push=true hub.push_gguf_repo=true hub.private=false
7
8 # 2. Nebium-Medium (345M parameters, 24 layers, d_model=1024, batch=2, accum=64)
9 python scripts/train.py --config-name=kaggle_medium \
10     data.hf_dataset.repo_id=nabin2004/nebium-lichess-uci \
11     hub.repo_id=nabin2004/nebium-medium \
12     hub.gguf_repo_id=nabin2004/nebium-medium-gguf \
13     hub.push=true hub.push_gguf_repo=true hub.private=false
14
15 # 3. Nebium-Large (762M parameters, 36 layers, d_model=1280, batch=1, accum=128,
16     8-bit AdamW)
17 python scripts/train.py --config-name=kaggle_large \
18     data.hf_dataset.repo_id=nabin2004/nebium-lichess-uci \
19     hub.repo_id=nabin2004/nebium-large \
20     hub.gguf_repo_id=nabin2004/nebium-large-gguf \
21     hub.push=true hub.push_gguf_repo=true hub.private=false

```

**Listing 6:** Command-line training invocations for all three model tiers.

## A.6 Hydra Configuration Files (YAML)

```

1 defaults:
2   - model: nebium_117m
3   - training: kaggle_117m
4   - data: kaggle
5   - logging: wandb
6   - hub: huggingface
7   - _self_
8
9 smoke_test: false
10
11 hydra:
12   run:
13     dir: /kaggle/working/outputs/nebium-small-${now:%Y-%m-%d_%H-%M-%S}

```

**Listing 7:** Root platform configuration for Nebium-Small in configs/kaggle\_small.yaml.

```

1 # Nebium-Small (117M) -- GPT-2 Small scale (Chinchilla-optimal: ~2.3B tokens)
2 optimizer: adamw
3 learning_rate: 3e-4
4 weight_decay: 0.1
5 lr_scheduler: cosine
6 warmup_steps: 2000
7 batch_size: 4
8 epochs: 20
9 gradient_accumulation_steps: 32          # Effective batch: 128 sequences
10 mixed_precision: fp16
11 gradient_clipping: 1.0
12 early_stopping_patience: 5
13 eval_samples: true
14 eval_puzzles: true

```

```

15 puzzle_path: "data/fixtures/puzzles.jsonl"
16 sample_prompts:
17   - ""
18   - "e2e4"
19   - "d2d4"
20 sample_max_moves: 20
21 sample_temperature: 0.7
22 export_gguf: true
23 gguf_precision: fp16
24 generate_report: false

```

**Listing 8:** Training optimization configuration for Nebium-Small in configs/training/kaggle\_117m.yaml.

```

1 # Nebium-Medium (345M) -- GPT-2 Medium scale (Chinchilla-optimal: ~6.9B tokens)
2 optimizer: adamw
3 learning_rate: 2e-4
4 weight_decay: 0.1
5 lr_scheduler: cosine
6 warmup_steps: 2000
7 batch_size: 2
8 epochs: 20
9 gradient_accumulation_steps: 64          # Effective batch: 128 sequences
10 mixed_precision: fp16
11 gradient_clipping: 1.0
12 early_stopping_patience: 5
13 eval_samples: true
14 eval_puzzles: true
15 puzzle_path: "data/fixtures/puzzles.jsonl"
16 sample_prompts:
17   - ""
18   - "e2e4"
19   - "d2d4"
20 sample_max_moves: 20
21 sample_temperature: 0.7
22 export_gguf: true
23 gguf_precision: fp16
24 generate_report: false

```

**Listing 9:** Training optimization configuration for Nebium-Medium in configs/training/kaggle\_345m.yaml.

```

1 # Nebium-Large (762M) -- GPT-2 Large scale (Chinchilla-optimal: ~15.2B tokens)
2 optimizer: adamw_8bit
3 learning_rate: 1.5e-4
4 weight_decay: 0.1
5 lr_scheduler: cosine
6 warmup_steps: 3000
7 batch_size: 1
8 epochs: 3
9 gradient_accumulation_steps: 128        # Effective batch: 128 sequences
10 mixed_precision: fp16
11 gradient_checkpointing: true
12 disable_data_parallel: true
13 gradient_clipping: 1.0

```

```

14 early_stopping_patience: 5
15 eval_samples: true
16 eval_puzzles: true
17 puzzle_path: "data/fixtures/puzzles.jsonl"
18 sample_prompts:
19   - ""
20   - "e2e4"
21   - "d2d4"
22 sample_max_moves: 20
23 sample_temperature: 0.7
24 export_gguf: true
25 gguf_precision: fp16
26 generate_report: false

```

**Listing 10:** Training optimization configuration for Nebium-Large in configs/training/kaggle\_762m.yaml.

```

1 _target_: src.models.transformer.nebium.Nebium
2 vocab_size: 5000
3 d_model: 1280
4 n_heads: 20
5 n_layers: 36
6 dropout: 0.1
7 positional_encoding: rope
8 attention_type: standard
9 activation: swiglu
10 norm: rmsnorm
11 max_seq_len: 1024
12 gradient_checkpointing: true

```

**Listing 11:** Architectural configuration for Nebium-Large in configs/model/nebium\_762m.yaml.

## A.7 Weights & Biases Diagram and Figure Reproduction Script

Listing 12 provides the exact CLI command to reproduce all diagrams using the WandB API.

```

1 # Option A: Supply API key via CLI argument
2 python report/latex/scripts/generate_paper_figures.py --api-key <
   YOUR_WANDB_API_KEY>
3
4 # Option B: Authenticate via environment variable
5 export WANDB_API_KEY="<YOUR_WANDB_API_KEY>"
6 python report/latex/scripts/generate_paper_figures.py

```

**Listing 12:** Executing the reproducible figure generation script with WandB credentials.

## B Open-Source Model Checkpoints and Artifact Repositories

To support open verification and reproducibility of the empirical findings reported in this work, all trained Nebium model checkpoints, quantized edge binaries, and tokenized chess corpora are published on the Hugging Face Hub.

The collection is accessible at:

<https://hf.co/collections/nabin2004/nebium-transformer-4-chess>

### B.1 Published Model Variants and Quantized Binaries

Table 6 lists the published model checkpoints across parameter scaling tiers, distinguishing standard PyTorch weights from edge-optimized GGUF binaries.

**Table 6:** Published Nebium model weights and quantized edge deployment binaries on Hugging Face.

| Model ID (nabin2004/)              | Params | Format       | Architecture               | Deployment Target & Notes                                 |
|------------------------------------|--------|--------------|----------------------------|-----------------------------------------------------------|
| <a href="#">nebium</a>             | 28.4M  | PyTorch FP16 | $L = 6, d = 512, H = 8$    | Base model for latency benchmarking                       |
| <a href="#">nebium-small</a>       | 117M   | PyTorch FP16 | $L = 12, d = 768, H = 12$  | Primary scaling tier for sequence loss analysis           |
| <a href="#">nebium-small-gguf</a>  | 117M   | GGUF Q4_K_M  | $L = 12, d = 768, H = 12$  | Edge binary (88.1 MB) for llama.cpp CPU inference         |
| <a href="#">nebium-medium</a>      | 345M   | PyTorch FP16 | $L = 24, d = 1024, H = 16$ | Compute-optimal checkpoint (val PPL 10.76, val loss 2.38) |
| <a href="#">nebium-medium-gguf</a> | 345M   | GGUF Q4_K_M  | $L = 24, d = 1024, H = 16$ | 4-bit block quantized binary for desktop CPUs             |
| <a href="#">nebium-large</a>       | 762M   | PyTorch FP16 | $L = 36, d = 1280, H = 20$ | Maximum capacity tier trained with 8-bit AdamW            |
| <a href="#">nebium-large-gguf</a>  | 762M   | GGUF Q4_K_M  | $L = 36, d = 1280, H = 20$ | 4-bit quantized binary for memory-constrained devices     |

### B.2 Published Datasets and UCI Token Splits

The training, validation, and tactical evaluation corpora are indexed in Table 7.

### B.3 Hugging Face Quickstart and Inference Invocation

Listing 13 demonstrates how to load Nebium model weights and tokenizer directly from the Hugging Face Hub for autoregressive move generation.

```

1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 import torch
3

```

**Table 7:** Published training datasets and tokenized chess corpora on the Hugging Face Hub.

| Dataset ID (nabin2004/)                | Records    | Format     | Contents and Function                                                     |
|----------------------------------------|------------|------------|---------------------------------------------------------------------------|
| <a href="#">ChessDataset</a>           | >2M games  | PGN / JSON | Competitive game collection filtered for rating > 2000                    |
| <a href="#">nebiu-lichess-2013-uci</a> | 1.2M plies | UCI text   | Standard games from Lichess 2013 opening benchmark                        |
| <a href="#">nebiu-lichess-uci</a>      | Full split | BPE tokens | Domain-tokenized UCI sequences ( $V = 2,056$ ) with train/val/test splits |

```

4 # Load domain tokenizer and model weights from Hugging Face Hub
5 repo_id = "nabin2004/nebiu-small"
6 tokenizer = AutoTokenizer.from_pretrained(repo_id)
7 model = AutoModelForCausalLM.from_pretrained(repo_id, torch_dtype=torch.float16)
8 model.eval()
9
10 # Encode Italian Game opening prefix
11 prompt = "e2e4 e7e5 g1f3 b8c6 f1c4"
12 inputs = tokenizer(prompt, return_tensors="pt")
13
14 # Autoregressively sample next move token
15 with torch.no_grad():
16     outputs = model.generate(**inputs, max_new_tokens=4, temperature=0.7)
17 continuation = tokenizer.decode(outputs[0], skip_special_tokens=True)
18 print(f"Game prefix: {prompt}")
19 print(f"Generated continuation: {continuation}")

```

**Listing 13:** Loading Nebiu-Small weights and tokenizer from the Hugging Face Hub.

## B.4 Detailed Hyperparameter Specifications Across Model Tiers

Tables 8, 9, and 10 provide the full architectural, optimization, and hardware configurations for each tier in the Nebiu transformer family. The verbatim Hydra YAML source configurations and automated multi-tier training scripts are documented in Section A.6 and Section A.5 of the Code Appendix.

**Table 8:** Nebium-Small (117M) architectural and training configuration (`nebiu_117m.yaml`, `kaggle_117m.yaml`).

| Hyperparameter                                      | Value                                                    |
|-----------------------------------------------------|----------------------------------------------------------|
| <i>Architecture</i>                                 |                                                          |
| Layers ( $L$ ) / Hidden dim ( $d$ ) / Heads ( $H$ ) | 12 / 768 / 12                                            |
| Head dim ( $d_k$ ) / SwiGLU dim ( $d_{ff}$ )        | 64 / 2,048                                               |
| Max sequence length                                 | 1,024 tokens                                             |
| Gradient checkpointing                              | Disabled                                                 |
| <i>Training</i>                                     |                                                          |
| Optimizer                                           | AdamW ( $\beta_1=0.9$ , $\beta_2=0.95$ , $\lambda=0.1$ ) |
| Peak LR / Schedule                                  | $3.0 \times 10^{-4}$ / Cosine, 2,000-step warmup         |
| Per-device batch / Accumulation steps               | 4 / 32 (effective: 128 seqs)                             |
| Mixed precision / Grad clip                         | FP16 AMP / $\ \mathbf{g}\ _2 \leq 1.0$                   |
| Epochs / Early-stop patience                        | 20 / 5                                                   |
| Chinchilla-optimal tokens                           | $\approx 2.3\text{B}$                                    |
| <i>Outputs</i>                                      |                                                          |
| GGUF export                                         | Q4_K_M, FP16 (88.1 MB)                                   |
| Hugging Face                                        | nabin2004/nebiu-small                                    |

**Table 9:** Nebium-Medium (345M) architectural and training configuration (`nebiu_345m.yaml`, `kaggle_345m.yaml`).

| Hyperparameter                                      | Value                                                    |
|-----------------------------------------------------|----------------------------------------------------------|
| <i>Architecture</i>                                 |                                                          |
| Layers ( $L$ ) / Hidden dim ( $d$ ) / Heads ( $H$ ) | 24 / 1,024 / 16                                          |
| Head dim ( $d_k$ ) / SwiGLU dim ( $d_{ff}$ )        | 64 / 2,736                                               |
| Max sequence length                                 | 1,024 tokens                                             |
| Gradient checkpointing                              | Disabled                                                 |
| <i>Training</i>                                     |                                                          |
| Optimizer                                           | AdamW ( $\beta_1=0.9$ , $\beta_2=0.95$ , $\lambda=0.1$ ) |
| Peak LR / Schedule                                  | $2.0 \times 10^{-4}$ / Cosine, 2,000-step warmup         |
| Per-device batch / Accumulation steps               | 2 / 64 (effective: 128 seqs)                             |
| Mixed precision / Grad clip                         | FP16 AMP / $\ \mathbf{g}\ _2 \leq 1.0$                   |
| Epochs / Early-stop patience                        | 20 / 5                                                   |
| Chinchilla-optimal tokens                           | $\approx 6.9\text{B}$                                    |
| <i>Outputs</i>                                      |                                                          |
| GGUF export                                         | Q4_K_M, FP16                                             |
| Hugging Face                                        | nabin2004/nebiu-medium                                   |

**Table 10:** Nebium-Large (762M) architectural and training configuration (`nebiu762m.yaml`, `kaggle_762m.yaml`). Empirical results from WandB run `b87goptx` (3 epochs on 12.0M tokens).

| Hyperparameter                                                  | Value                                                   |
|-----------------------------------------------------------------|---------------------------------------------------------|
| <i>Architecture</i>                                             |                                                         |
| Layers ( $L$ ) / Hidden dim ( $d$ ) / Heads ( $H$ )             | 36 / 1,280 / 20                                         |
| Head dim ( $d_k$ ) / SwiGLU dim ( $d_{ff}$ )                    | 64 / 3,416                                              |
| Max sequence length                                             | 1,024 tokens                                            |
| Gradient checkpointing                                          | <b>Enabled</b>                                          |
| <i>Training</i>                                                 |                                                         |
| Optimizer                                                       | 8-bit AdamW ( <code>adamw_8bit</code> , $\lambda=0.1$ ) |
| Peak LR / Schedule                                              | $1.5 \times 10^{-4}$ / Cosine, 3,000-step warmup        |
| Per-device batch / Accumulation steps                           | 1 / 128 (effective: 128 seqs)                           |
| Data parallel                                                   | Disabled ( <code>disable_data_parallel: true</code> )   |
| Mixed precision / Grad clip                                     | FP16 AMP / $\ \mathbf{g}\ _2 \leq 1.0$                  |
| Epochs / Early-stop patience                                    | 3 (early run) / 5                                       |
| Chinchilla-optimal tokens                                       | $\approx 15.2\text{B}$                                  |
| <i>Observed Results (WandB: <code>b87goptx</code>, Epoch 3)</i> |                                                         |
| Train loss                                                      | 6.3074 nats                                             |
| Validation loss                                                 | <b>6.0707 nats</b> (checkpoint minimum)                 |
| Validation perplexity                                           | $\approx 435$                                           |
| Board-legal move rate                                           | $\approx 80\%$                                          |
| <i>Outputs</i>                                                  |                                                         |
| GGUF export                                                     | Q4_K_M, FP16                                            |
| Hugging Face                                                    | <a href="#">nabin2004/nebiu-large</a>                   |